

MSc Mathematics
Track: Statistics and Probability Theory

Master thesis

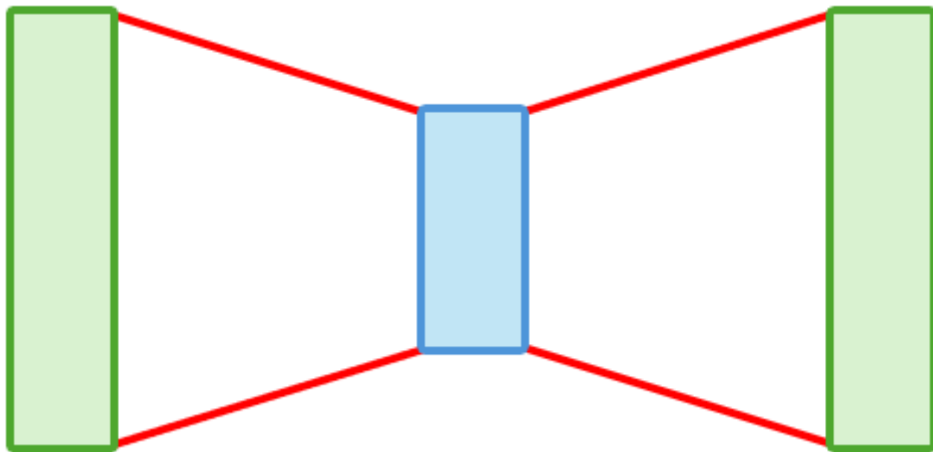
Autoencoders as Variations of PCA and an Application to Sparse Data Using the ReLU Activation

by
Mathys Goosen

June 30, 2025

Supervisor: dr. Paulo de Andrade Serra

Second examiner: dr. Wessel van Wieringen



Department of Mathematics
Faculty of Sciences

Abstract

Autoencoders are neural networks constructed to perform feature extraction. The implementation of other feature extraction methods as autoencoders would allow for the possibility of leveraging the broader neural network framework to find more efficient implementations as well as using specific constructions of neural networks to extend the concept of such feature extraction methods. Here we explore the known connections between PCA and autoencoders to find a novel construction of a linear autoencoder that is able to exactly replicate PCA as well as a variation thereof, Sparse PCA. This construction is made possible by the use of skew-symmetric matrices and matrix exponentials to enforce the required orthogonality in the trainable weight matrices of the autoencoders. By reproducing results found regarding the phenomenon of superposition in [31], we demonstrate the ability of adding non-linear ReLU activation in autoencoders to improve on the performance of PCA by finding more complex relationships between observations, for varying levels of sparsity in the corresponding dataset. We conclude with a summary of the connection between autoencoders for sparse datasets and the field of compressive sensing.

Title: Autoencoders as Variations of PCA and an Application to Sparse Data Using the ReLU Activation

Author: Mathys Goosen, m.c.goosen@student.vu.nl, 2600867

Supervisor: dr. Paulo de Andrade Serra

Second examiner: dr. Wessel van Wieringen

Date: June 30, 2025

Department of Mathematics

VU University Amsterdam

de Boelelaan 1081, 1081 HV Amsterdam

<http://www.math.vu.nl/>

Contents

1	Introduction	4
2	Literature Review	7
2.1	Regression	7
2.1.1	High Dimensionality in Regression	8
2.1.2	Regression Methods	13
2.2	Feature Extraction	16
2.2.1	Principal Component Analysis	17
2.2.2	Ridge PCA	20
2.2.3	Sparse PCA	20
2.3	Neural Networks	21
2.3.1	PCA as Neural Networks	22
2.3.2	Autoencoders	23
2.3.3	Optimization	24
3	Results	27
3.1	PCA as Autoencoders	27
3.1.1	Linear Autoencoder	28
3.1.2	Ridge and Sparse Linear Autoencoders	30
3.1.3	Numerical Illustration	33
3.2	Superposition	38
3.2.1	Linear Toy-Model	39
3.2.2	ReLU Toy-Model	44
3.2.3	Compressive Sensing	48
4	Discussion	51
5	Future Work	54
6	Appendix	56
	Bibliography	60

1 Introduction

The ability to reduce the dimensionality of datasets is crucial in statistics and machine learning to mitigate two flavors of the curse of dimensionality, namely computational complexity and the exponential relationship between the dimension and required observations needed to make a good estimate [1, 7]. The resulting effect of dimensionality reduction is two-fold; the improvement of model performance and interpretability [2].

The two main approaches of dimensionality reduction are feature selection and feature extraction [2]. Feature selection refers to choosing a subset of the most informative features of the original dataset. Feature extraction, the main approach we will focus on, refers to deriving new lower-dimensional features from the original data in an attempt to capture essential patterns and relationships.

Linear methods of feature extraction are relatively straightforward in that they transform the original dataset as functions of the features from the original dataset. Linear methods such as PCA (Principal Component Analysis) and LDA (Latent Discriminant Analysis) have more traditional roots in statistics and have thoroughly been explored [3]. When the dimensionality of the dataset is not too large, linear methods have been proven to be computationally effective. They also often provide interpretable results, simplify the understanding of the importance of each new feature and are effective when the underlying relationships of the dataset are approximately linear [15].

When the underlying relationships of the dataset are not linear, these methods become less effective. Additionally, the computational efficiency of these methods do not scale with the increase of dimensionality of the dataset. The increase in dimensionality also tends to result in overfitting and consequently the loss of interpretability of the features, especially in a *high-dimensional* setting: when the dimension of the dataset greatly exceeds the number of observations [19].

Non-linear feature extraction methods are able to overcome the limited ability linear methods have to capture complex non-linear relationship in the dataset. Kernel methods are often used to accomplish this. By increasing the dimensionality of the extracted features, one can find more complex non-linear relationships that are not apparent in the space of linear transformed features. Kernel methods also perform well in a high-dimensional setting [4]. However, increasing the dimensionality of the features directly contradicts the idea of dimensionality reduction, namely to find a lower dimensional representation that manages to store most of the underlying information in a dataset.

Manifold based feature extraction methods such as LLE (Locally Linear Embedding) and t-SNE (t-distributed Stochastic Neighbor Embedding) [5, 6], attempt to find a non-

linear map that embeds the dataset in a lower dimensional space that preserves the structure of the data. However, these methods often make the trade off of preserving local structure at the cost of losing the global structure of data. They are also extremely sensitive to the choice of hyperparameters [1].

A different feature extraction method that can be implemented both linearly and non-linearly is deep learning and specifically neural networks. The main benefit of using neural networks is its modularity [21]. One can implement a neural network to perform feature extraction, but also specific tasks such as regression and classification. These separate neural networks can then easily be combined to perform both tasks as a singular neural network.

A specific class of neural networks used in feature extraction is Autoencoders [25]. The main distinction between Autoencoders and other neural networks used in feature extraction, such as RNN's (Recurrent Neural Networks) and CNN's (Convolutional Neural Networks) are that Autoencoders are used for unsupervised learning. Meaning that Autoencoders can perform meaningful feature extraction with unlabeled data, whereas other neural networks usually require labeled data to effectively perform feature extraction.

Autoencoders are tasked to find two (possibly) non-linear transformations. One from the space spanned by the original dataset to some, often lower dimensional, feature space. The other from the feature space back to the original space with the goal to reconstruct a dataset via these two transformations [25]. The quality of these transformations are measured by a reconstruction error, that measures the information lost in the reconstruction of the dataset using the learned transformations. The flexibility and benefit of autoencoders lie in the fact that we are free to make constraints and choose required properties of these transformations as desired. We can also define the reconstruction loss to measure any variation of information lost in the reconstruction. Furthermore autoencoders are also able extract features without the need for manual pre-processing [21].

The flexibility of autoencoders results in a broad array of applications. The ability of autoencoders to effectively encode data in a lower dimensional representation allows for applications in data compression, storage and transmission. Autoencoders can also use the reconstruction error to effectively detect anomalies and outliers in datasets. Autoencoders similarly excel in denoising of data to improve data quality [21]. However the most prevalent application of autoencoders in recent times, is its apparent effective learning of semantic embedding in Natural Language Processing, where autoencoders seemingly make use of the sparsity of input data to find more meaningful complex relationships in data comprising of natural language [31].

In this thesis we explore the possibilities that present themselves as a result of the generality that autoencoders provide. Specifically, in Section 2.2.1 we explore how PCA is defined and redefined as different loss functions that need to be optimized. In Section 2.3 we then formally define autoencoders and how they are optimized, with the goal

of finding a network structure and suitably defined reconstruction error such that this specific autoencoder exactly performs PCA.

In Section 3.1 we show that this is indeed possible as well as that this specific autoencoder automatically performs the necessary pre-processing that needs to be done to perform PCA. We are then also able to extend this reformulation to Sparse PCA.

The natural extension of an autoencoder designed to mimic PCA, is to add non-linearities in the transformations to and from the feature space. In Section 3.2 we explore an interesting property that was found in the context of Natural Language Processing and the interpretability of these models. It turns out that in the constraints of a specific toy-model, the addition of a ReLU non-linearity to a version of our defined "PCA autoencoder" allows for learning of more complex relationships between data when the sparsity of input dataset is increased.

We round off the main results in Section 3.2.3 by mentioning the connection between autoencoders applied to sparse data and the theory of compressive sensing, the reconstruction of sparse signals from measured data. This connection allows us, under strict conditions, to determine suitable structures of autoencoders that will accurately reproduce sparse data.

The specific contributions of this thesis are:

- PCA is known to connect to specific autoencoders [19, 20], in Section 3.1 we summarize these connections as well as expand upon known properties of these specific autoencoders.
- The key to formulating an autoencoder as a PCA problem is the implementation of the hard orthogonality constraint of the PCA loading vectors. We show in Section 3.1.2 that this constraint can be implemented to autoencoders by using skew-symmetric matrices and matrix exponentials.
- PCA and Sparse PCA can be expressed as respectively ridge regression and elastic net problems [16]. In Section 3.1.2, we construct autoencoders that are able to solve these problems. The resulting autoencoders are then successfully able to implement PCA and Sparse PCA. From these autoencoders, we can recover each individual loading vector as well as each individual principal component from PCA via respectively, normalized versions of the encoder weight matrix and the encoder function itself. This improves upon existing autoencoders that are only able to recover rotated versions of the loading vectors and principal components.
- A connection between autoencoders and the phenomenon of superposition in sparse data models has been presented in [31]. In Section 3.2, we reproduce these results and explicitly state properties that are only implicitly mentioned in [31].
- It has been proposed in [31] to connect autoencoders for sparse datasets to the field of compressive sensing. We introduce and discuss this proposed connection, describing the limitations as well as the main challenges that need to be overcome to apply this connection to the evaluation of practical examples.

2 Literature Review

2.1 Regression

With this Literature Review, we introduce the problem of regression, partly as a motivation for the need of feature extraction and partly to introduce tools used in regression that can be applied to PCA and autoencoders later on.

This section is mainly based on the material from Chapters 4 and 5 of [7].

We start by considering an experiment of n pairs of samples $(x_1, Y_1), \dots, (x_n, Y_n)$. Here we call Y the *response* and x the *observation* (x is often also called the *covariate* or a *feature*). In our experiment we assume that Y and x are related by the equations

$$Y_i = r(x_i) + \epsilon_i, \quad \mathbb{E}(\epsilon_i) = 0, i = 1, \dots, n, \quad (2.1)$$

where $r \in \mathcal{F}$ is called the *regression function* and $\mathcal{F}$ is some class of (possibly infinite dimensional) functions. The goal of our experiment is to estimate $r(x)$, under weak assumptions, by the estimator $\hat{r}_n$ called the *smoother*. In (2.1) we assume the observations x_i to be fixed. We could however assume that these observations are random. Then our samples are written as $(X_1, Y_1), \dots, (X_n, Y_n)$ and $r(x)$ can be interpreted as

$$r(x) = \mathbb{E}(Y|X = x),$$

i.e. the mean of Y conditional on the observation that $X = x$.

Due to the generality of the problem described above, there are many choices for the smoother. Often, the smoother depends on some parameter h , called the *smoothing parameter*, that determines how much smoothing we want to apply to our estimate of $r(x)$. We denote this dependency on h as $\hat{r}_{h,n}(x)$. One way to evaluate the smoother is through the *risk*. The risk at a point x is defined as the average of the loss between a function $f(x)$ and an estimate $\hat{f}_n(x)$ and is denoted by

$$R(f(x), \hat{f}_n(x)) = \mathbb{E}_x \left(\mathcal{L}(f(x), \hat{f}_n(x)) \right). \quad (2.2)$$

Here $f(x)$ is a unknown deterministic function, $\hat{f}_n(x)$ the random variable, depending on the observed samples, which the expectation is taken over and $\mathcal{L}$ a chosen loss function.

If we choose $\mathcal{L}$ as the *squared error* loss function defined as

$$\mathcal{L}(f(x), \hat{f}_n(x)) = (f(x) - \hat{f}_n(x))^2,$$

the risk is then called the *Mean Squared Error* (MSE) and can be defined as

$$R(f(x), \hat{f}_n(x)) = \mathbb{E}_x \left((f(x) - \hat{f}_n(x))^2 \right). \quad (2.3)$$

Ideally one would then like to choose $\hat{f}_n(x)$ such that the MSE is minimized. Furthermore, it can be shown that the MSE can be decomposed into a term that describes the bias and a term that describes the variance of the smoother as follows

$$R(f(x), \hat{f}_n(x)) = \underbrace{\left(\mathbb{E}_x(\hat{f}_n(x)) - f(x) \right)^2}_{\text{bias}} + \underbrace{\mathbb{V}_x(\hat{f}_n(x))}_{\text{variance}}.$$

In the context of regression, we want to summarize the MSE over all the observations $x_1, \dots, x_n$ in terms of $r(x)$ and the smoother $\hat{r}_{h,n}$ with regard to a choice for h . To do this, we make use of the *average mean squared error* (AMSE) denoted by

$$R(r, \hat{r}_{h,n}) := R(h) = \mathbb{E}_x \left(\frac{1}{n} \sum_{i=1}^n (r(x_i) - \hat{r}_{h,n}(x_i))^2 \right). \quad (2.4)$$

The main challenge in choosing a suitable smoother is to determine how much smoothing to do. When the data is over-smoothed, the bias term is large and the variance is small. When the data is under-smoothed the opposite is true. This is called the *bias-variance trade-off*. Minimizing the MSE corresponds to balancing bias and variance.

2.1.1 High Dimensionality in Regression

An additional challenge that arises when choosing a suitable smoother is when the dimensionality p of the observations $x_i \in \mathbb{R}^p$ for $i \in \{1, \dots, n\}$, is large. Specifically, when the dimension of an observation x_i is much larger than n the number of samples. This specific setting, when $p \gg n$, is called *high-dimensionality* [11].

To introduce how the dimensionality of $x_1, \dots, x_n$ influences the choice of smoothers, we start with an example that involves a kernel estimator of the smoother $\hat{r}_{h,n}$.

Kernel Regression

A specific example where the dimensionality of the observations $x_1, \dots, x_n$ plays a role in the bias-variance trade-off, lies in the Nadarya-Watson kernel estimator.

Definition 2.1 (Nadaraya-Watson Estimator). We can define the Nadaraya-Watson kernel estimator as the linear smoother

$$\hat{r}_{h,n}(x) = \sum_{i=1}^n h_i(x) Y_i, \quad \text{where } h_i(x) = \frac{K_p(\frac{x_i - x}{\mathbf{h}})}{\sum_{j=1}^n K_p(\frac{x_j - x}{\mathbf{h}})},$$

for $x \in \mathbb{R}^p$ and where $x_i \in \mathbb{R}^p, Y_i \in \mathbb{R}$ is the i 'th observation and response sample pair, K_p a kernel function on $\mathbb{R}^p$ (Appendix A) and $\mathbf{h} = (h_1, \dots, h_p)$ a chosen smoothing parameter¹.

We see that by choosing an appropriate value for $h_i \in \mathbf{h}$ we can control the amount of influence $x_{i,j}$, the j 'th entry of an observation x_i , has on our estimation of $r(x)$. More concretely if we choose for example $(h_1, \dots, h_k)$ to be small and $(h_{k+1}, \dots, h_p)$ to be large, we attempt to estimate $r(x)$ mostly from the information in the first k entries of the observations $x_1, \dots, x_n$ and smoothing out the information of the remaining $p - k$ entries. This essentially amounts to some sort of feature selection, as mentioned in the introduction, without explicitly reducing the dimension of the observations.

A big drawback however, when using the Nadaraya-Watson Estimator as a smoother, arises when we increase the dimension of the observations in our experiment. It turns out that the risk increases exponentially along with the increase of p , the dimension of the observations $x_1, \dots, x_n$.

To show this phenomenon, we choose $\mathbf{h} = (h, \dots, h)$ and find an expression, under mild regularity conditions, of the risk of the Nadaraya-Watson kernel estimator corresponding to the *mean integrated square error*, which is defined as

$$R(h) = \int_{\mathcal{D} \subset \mathbb{R}^p} \mathbb{E}_x (r(x) - \hat{r}_{h,n}(x))^2 dx,$$

This expression of $R(h)$ is consequently decomposed into a bias term and a variance term as follows

Theorem 2.2 (Theorem 2 of [9]). *Under regularity conditions and for $\hat{r}_{h,n}(x)$ the Nadarya-Watson kernel estimator, if $h \rightarrow 0$ and $nh^p \rightarrow \infty$ as $n \rightarrow \infty$ we have*

$$R(h) = \underbrace{h^4 \int_{\mathcal{D}} \frac{\gamma^2(x)}{g(x)^2} dx}_{\text{bias}} + \underbrace{\frac{1}{nh^p} \int_{\mathcal{D}} \frac{\sigma^2(x)}{g(x)} dx \cdot \int K_p^2(u) du}_{\text{variance}} + o_p \left(h^4 + \frac{1}{nh^p} \right),$$

where $\sigma^2(x) = \mathbb{E}(Y^2 | X = x) - r(x)^2$ the variance of the response Y conditional on $X = x$, $g(x)$ the density of x and $\gamma(x)$ a mixture term of the 1st and 2nd partial derivatives of $r(x)$ ².

¹Here, for vectors $\mathbf{a}, \mathbf{b}$ we define $\mathbf{a} - \mathbf{b} = (a_1 - b_1, \dots, a_p - b_p)$ and $\mathbf{a}/\mathbf{b} = (a_1/b_1, \dots, a_p/b_p)$

²See Appendix A for all regularity conditions and the full expression of $\gamma(x)$.

It follows from this theorem that the risk is bounded as

$$R(h) = \underbrace{O_p(h^4)}_{\text{bias}} + \underbrace{O_p\left(\frac{1}{nh^p}\right)}_{\text{variance}} + o_p\left(h^4 + \frac{1}{nh^p}\right)$$

when $n \rightarrow \infty$.

What we see from this theorem is that indeed by increasing h we can reduce the variance of the risk at the cost of introducing a bias and vice versa. It turns out that the minimal risk is of order $n^{-4/(p+4)}$ for some optimal $h_* \approx Cn^{-1/(p+4)}$, with constant $C > 0$, that minimizes $R(h)$. This then implies that, even for an optimal h_* , the risk still increases exponentially with p . We can try to reduce the risk by increasing n , however as we will see, in general, if we want to bound the MSE, then n also increases exponentially with p .

Curse of Dimensionality

A more general problem that occurs when estimating the regression function $r(x)$ (and smoothing methods in general) is the curse of dimensionality. Informally, this refers to the fact that the estimation of $r(x)$ quickly increases in difficulty when the dimension of the support (Appendix A) of $r(x)$ in our experiment increases, i.e. when $x \in \mathbb{R}^p$ and p is large [8].

To us, there are two relevant versions of this curse. Firstly, the computational curse of dimensionality, referring to the computational burden of some methods than can increase exponentially with dimension. The second version, which is called the *statistical curse of dimensionality*, refers to the fact that if $x \in \mathbb{R}^p$, then we need a sample size n that grows exponentially with p to ensure a good estimation of $r(x)$ in terms of the MSE.

It can be shown that typically, in the case of the model described in (2.1), we have that

$$R(r, \hat{r}_{h,n}) \approx \frac{c}{n^{2\alpha/(2\alpha+p)}}$$

for some $c > 0$ and where α is the number of derivatives of $r(x)$. Specifically, in (2.1), we assumed that $\alpha = 2$.

If we then want the MSE to be small, say for some $\delta > 0$, we can set $R(r, \hat{r}_{h,n}) = \delta$ and solve for n . We would then find that

$$n \propto \left(\frac{c}{\delta}\right)^{p/4}$$

and see that n grows exponentially with dimension p [8].

This can be explained by the fact that estimating the smoother involves using data points in a local neighborhood of the point x . However, typically, in a high-dimensional setting, where $p \gg n$, the data is very sparse (most entries of x are zero), implying that local neighborhoods of the entire dataset $x_1, \dots, x_n$ contain very few points.

Sparsity

Just as we use our choice of the smoothing parameter h to introduce structure into our estimation of $r(x)$ we can use the assumption of sparsity in our smoother to find a better estimation of $r(x)$. It turns out that sparsity, especially in a high-dimensional setting ($p \gg n$), is a convenient structure of a dataset that can be harnessed to partially overcome the statistical curse of dimensionality. The field related to methods involving sparse datasets is called *compressive sensing*.

This section is based on the material from Chapters 1-3 of [10].

The notion of sparsity is related to the number of non-zero entries in a vector z .

Definition 2.3. A vector $z \in \mathbb{R}^N$, for $N \geq s$, is s -sparse if

$$\|z\|_0 \leq s,$$

where $\|\cdot\|_0$ is the ℓ_0 norm.

Therefore z is s -sparse if there are at most s nonzero entries in z . It turns out that we can exploit this sparsity structure of a vector to efficiently find unknown vectors $z \in \mathbb{R}^N$, that are assumed to be sparse, from known measurements $y \in \mathbb{R}^m$ (that might not be sparse themselves). The field of *compressive sensing* is devoted to exactly this goal.

Compressive sensing makes use of sparsity to find efficient algorithms that aim to recover an unknown signal $z \in \mathbb{R}^N$ from measurements $y \in \mathbb{R}^m$ that are related to z via a $m \times N$ matrix A as follows

$$y = Az. \tag{2.5}$$

Here A represents the linear measurement process of the signal z . If we were to solve this linear system for specifically $m < N$ (high-dimensionality) and do not make any assumption on the structure of z , then (2.5) would be underdetermined and there would be either an infinite amount of z 's that solve (2.5) or none at all. It turns out, if one were to assume that the underlying z is sparse, some efficient algorithms exist to solve (2.5) for some suitable choices for A such that we can reconstruct the signal z .

Specifically, if z_s is the unique s -sparse solution to $y = Az$, then we can reconstruct z_s as the unique solution to

$$\underset{z \in \mathbb{R}^N}{\text{minimize}} \quad \|z\|_0 \quad \text{subject to} \quad Az = y. \tag{2.6}$$

However, solving this problem is NP -hard in general and therefore useless for practical purposes. To solve this computation problem an adjusted algorithm is given that is also shown to be able to reconstruct a s -sparse z_s from (2.5) under certain conditions, only now much more efficiently. This algorithm is called *basis pursuit* or ℓ_1 -minimization and consists of finding the minimizer of the problem

$$\underset{z \in \mathbb{R}^N}{\text{minimize}} \quad \|z\|_1 \quad \text{subject to} \quad Az = y. \tag{2.7}$$

The success of this algorithm lies in the fact that we now have a convex optimization problem as well as the fact that the minimizer of (2.7), $z_1 \in \mathbb{R}^N$, is sparse. Specifically it holds that $\|z_1\|_0 \leq m$.

Practically, we cannot guarantee that the measurements y are precise. Therefore, it is more realistic to model the relationship of y, z and A in (2.5) as

$$y = Az + \varepsilon, \quad (2.8)$$

where ε is a noise vector such that $\|\varepsilon\|_2 \leq \eta$ for some $\eta > 0$. In this case we need to allow for some representation error η in (2.7). This transforms (2.7) into the adjusted *quadratically constrained basis pursuit* as follows

$$\text{minimize } \|z\|_1 \quad \text{subject to } \|Az - y\|_2 \leq \eta. \quad (2.9)$$

To relate compressive sensing to the problem of regression, we first define the *design matrix* as the $n \times p$ matrix consisting of the entries of each observation $x_i \in \mathbb{R}^p$ from our experiment in (2.1), i.e. $X_{(i,j)} := x_{i,j}$ where $i \in \{1, \dots, n\}$ and $j \in \{1, \dots, p\}$.

We can then set $y = (Y_1, \dots, Y_n)^T$ for each $Y_i \in \mathbb{R}^n$ the i 'th response from our experiment in (2.1), $\varepsilon = (\varepsilon_1, \dots, \varepsilon_n)^T$ for each $\varepsilon_i \in \mathbb{R}^n$ such that $\mathbb{E}(\varepsilon_i) = 0$ as defined in (2.1), as well as $\|\varepsilon\|_2 \leq \eta$. Then, if we let $A = X$, $z = \beta \in \mathbb{R}^p$, $m = n$ and $N = p$, we have reformulated (2.8) in terms of our experiment at the start of the chapter:

$$Y = X\beta + \varepsilon. \quad (2.10)$$

We have shown that if $n < p$ this linear system is underdetermined, with either infinite or no solutions. In practice only a number of the p components in X contribute to the measurements (response) Y , but it is unclear which of these components make a contribution. This results in β being sparse and determining a sparse β corresponds to selecting which of the p entries of the observations in X are relevant. This is called *model selection*. If we set $r(x_i) = x_i\beta_i$ then we have reformulated the regression problem (2.1) for a specific choice of r as a compressive sensing problem in the case of $n < p$. Therefore, we can use the compressive algorithm (2.9) to find a sparse $\hat{\beta}$ to estimate $r(x_i) = x_i\beta_i$ by the smoother $\hat{r}_n(x_i) = x_i\hat{\beta}_i$.

We will see below in Section 2.1.2 that there is an equivalence between the compressive sensing algorithm in (2.9) and a regression estimator called the LASSO. This equivalence allows us to use results from compressive sensing to bound the MSE associated with the LASSO, with specific conditions for X .

We have seen that without the additional assumption of sparsity in the observations $x_1, \dots, x_n$ of our experiment, in a high-dimensional setting, there is a need to reduce the dimensionality of the observations to attempt to circumvent the curse of dimensionality. This is where feature extraction might serve a purpose.

Before we introduce feature extraction in the form of the PCA, we first introduce specific regression techniques that attempt to estimate $r(x)$ and evaluate their performance when the observations $x_1, \dots, x_n$ are high-dimensional.

2.1.2 Regression Methods

We take a step back to look at the classical regression methods that have been developed to find an estimator for β as in (2.10) and see how they are able (either successfully or unsuccessfully) to deal with collinearity between observations or high-dimensionality, when $p \gg n$.

In the sections below, until stated otherwise, we define $Y = [Y_1, \dots, Y_n]^T \in \mathbb{R}^n$ the vector of responses of our experiment, $X \in \mathbb{R}^{n \times p}$ the design matrix defined entry-wise as $X_{(i,j)} = x_{i,j}$ the j 'th entry of the i 'th observation of our experiment, $\beta \in \mathbb{R}^p$ and $\varepsilon = [\varepsilon_1, \dots, \varepsilon_n]^T$ such that for each $i \in \{1, \dots, n\}$, $\mathbb{E}(\varepsilon_i) = 0$ and $\mathbb{V}(\varepsilon) = \sigma^2$.

Linear Regression

We start with the most basic regression model. On the assumption of the underlying relation of the model being

$$Y = X\beta + \varepsilon,$$

our goal is to find a suitable estimate for β , the *regression parameter*. The model set up in this way is called the *linear regression model*.

To estimate the regression parameter, we make use of the *minimum least squares estimator* defined as

$$\hat{\beta}_{MLS} = \arg \min_{\beta} \|Y - X\beta\|_2^2,$$

such that $\|\hat{\beta}_{MLS}\|_2^2 < \|\beta\|_2^2$ for all other β that minimize $\|Y - X\beta\|_2^2$. It can then be shown that

$$\hat{\beta}_{MLS} = (X^T X)^{-1} X^T Y, \tag{2.11}$$

assuming $(X^T X)^{-1}$ is well-defined [12].

The expectation and variance of $\hat{\beta}_{MLS}$ can be computed to find

$$\begin{aligned} \mathbb{E}(\hat{\beta}_{MLS}) &= \beta \\ \mathbb{V}(\hat{\beta}_{MLS}) &= \sigma^2 (X^T X)^{-1}. \end{aligned}$$

We immediately see that $\hat{\beta}_{MLS}$ is unbiased, resulting in the MSE of the estimator being

$$R(\beta, \hat{\beta}_{MLS}) = \mathbb{V}(\hat{\beta}_{MLS}) = \sigma^2 \text{tr}((X^T X)^{-1}),$$

where $\text{tr}(\cdot)$ is the *trace operator* (See Appendix A).

An immediate problem arises in the linear regression model when $(X^T X)^{-1}$ is not well-defined. An example of this comes in the form of *super-collinearity*, when the columns of X are linearly dependent. In the case of a high-dimensional model, when $p \gg n$, the $p \times p$ matrix $X^T X$ has at most rank n and is consequently singular. Therefore, $(X^T X)^{-1}$ is not well-defined and we are once again met with an underdetermined linear system with no unique solution for $\hat{\beta}_{MLS}$ such that there are possibly multiple $\hat{\beta}$'s that solve $X^T X \hat{\beta} = X^T Y$.

Rather than turning to compressive sensing in the case of super-collinearity we can instead make use of an alternative expression for $\hat{\beta}_{MLS}$. In the case of super-collinearity, it can be shown that

$$\hat{\beta}_{MLS} = (X^T X)^+ X^T Y,$$

where A^+ denotes the *Moore-Penrose inverse* (Appendix A) of a matrix A [11]. However, there are better methods to estimate β in the case of super-collinearity.

Ridge Regression

An alternate estimator for the regression parameter β that is used to deal with super-collinearity is presented in the form of the *ridge regression estimator*, defined as

$$\hat{\beta}_{ridge}(\lambda) := (X^T X + \lambda \mathbf{I}_p)^{-1} X^T Y,$$

for some chosen hyperparameter $\lambda > 0$ and $\mathbf{I}_p$ the $p \times p$ identity matrix. Adding the penalty $\lambda \mathbf{I}_p$ to $X^T X$ ensures that $X^T X + \lambda \mathbf{I}_p$ has no zero eigenvalues and consequently that $(X^T X + \lambda \mathbf{I}_p)^{-1}$ is well-defined.

We can alternatively express the ridge regression estimator in terms of the *singular value decomposition* or SVD (Appendix A) of a matrix. If we express the SVD of X as

$$X = U_x D_x V_x^T,$$

then $\hat{\beta}_{ridge}$ can be expressed as follows

$$\hat{\beta}_{ridge}(\lambda) = V_x (D_x^T D_x + \lambda \mathbf{I}_p)^{-1} D_x^T U_x^T Y.$$

We can compare this expression of $\hat{\beta}_{ridge}$ to that of $\hat{\beta}_{MLS}$ expressed in terms of the SVD of X :

$$\hat{\beta}_{MLS} = V_x (D_x^T D_x)^{-1} D_x^T U_x^T Y.$$

We see that for all $\lambda > 0$, it follows that $(D_x)_{jj}^{-1} > ((D_x)_{jj}^2 + \lambda)^{-1}$. Therefore the penalty in the ridge regression estimator shrinks the singular values based on the size of their effect. Through simulations, one can see which covariates are preferred, i.e. which covariates are shrunk less, among a set of covariates with similar effect sizes when using ridge regression. It can be seen that ridge regression prefers covariates that are strongly positively correlated as well as covariates with larger variance [11].

Another alternative expression for the ridge regression estimator, that will be the most relevant going forward, comes in the form of the minimization problem

$$\hat{\beta}_{ridge}(\lambda) = \arg \min_{\beta} \|Y - X\beta\|_2^2 + \lambda \|\beta\|_2^2,$$

which can be reformulated as a constrained estimation problem for a suitable $c(\lambda) > 0$:

$$\hat{\beta}_{ridge}(\lambda) = \arg \min_{\beta: \|\beta\|_2^2 \leq c(\lambda)} \|Y - X\beta\|_2^2.$$

Also, for $\hat{\beta}_{ridge}(\lambda)$ we can compute the expectation, variance and MSE:

$$\begin{aligned} \mathbb{E}(\hat{\beta}_{ridge}(\lambda)) &= \beta - \lambda(X^T X + \lambda \mathbf{I}_p)^{-1} \beta \\ \mathbb{V}(\hat{\beta}_{ridge}(\lambda)) &= \sigma^2 (X^T X + \lambda \mathbf{I}_p)^{-1} X^T X ((X^T X + \lambda \mathbf{I}_p)^{-1})^T \\ R(\beta, \hat{\beta}_{ridge}(\lambda)) &= \sigma^2 \text{tr}(W_\lambda (X^T X)^{-1} W_\lambda^T) + \beta^T (W_\lambda - \mathbf{I}_p)^T (W_\lambda - \mathbf{I}_p) \beta, \end{aligned}$$

where $W_\lambda = (X^T X + \lambda \mathbf{I}_p)^{-1} X^T X$. We immediately see that for any $\lambda \neq 0$, $\hat{\beta}_{ridge}(\lambda)$ is biased. However, to motivate the potential benefit of using biased regression estimators (and biased smoothers in general [3449]), it can be shown that there exists a $\lambda > 0$ such that

$$R(\beta, \hat{\beta}_{ridge}(\lambda)) < R(\beta, \hat{\beta}_{ridge}(0)) = R(\beta, \hat{\beta}_{MLS}),$$

meaning that for an appropriate choice of λ we can achieve better performance than the unbiased $\hat{\beta}_{MLS}$ in the MSE sense [13].

LASSO Regression

An alternative to Ridge Regression is LASSO Regression. Where the regression parameter is estimated by the *LASSO estimator* (or LASSO) which can be expressed as the following minimization problem [14]:

$$\hat{\beta}_{lasso}(\lambda) = \arg \min_{\beta} \|Y - X\beta\|_2^2 + \lambda \|\beta\|_1, \quad (2.12)$$

which can again similarly be reformulated as the constrained estimation problem:

$$\hat{\beta}_{lasso}(\lambda) = \arg \min_{\beta: \|\beta\|_1 \leq c(\lambda)} \|Y - X\beta\|_2^2.$$

A possible benefit of using the LASSO is that, as λ increases, a larger subset of components of $\hat{\beta}_{lasso}(\lambda)$ are set to zero. This would imply that if for some λ' it holds that $\hat{\beta}_{lasso}(\lambda')_j = 0$, the LASSO estimates that the j 'th covariate has no effect on the response and can be excluded from the model. This again leads to model selection, as described in Section 2.1.1. The main motivation for the addition of model selection in the context of regression was to improve the interpretability of models [16].

It is important to notice that the choice of λ does not always explicitly determine the number of non-zero covariates in the LASSO. Additionally, simulations with covariates that contribute equally to the response show that the LASSO prefers to select variables with the largest variance [11].

A disadvantage of the LASSO is that, unlike the ridge regression estimator, generally an analytic expression for $\hat{\beta}_{lasso}(\lambda)$ does not exist. An exception however arises when the design matrix is orthogonal. Then the LASSO can be expressed entry-wise as

$$\hat{\beta}_{lasso}(\lambda)_i = \left(|\hat{\beta}_i| - \frac{\lambda}{2} \right)_+ \text{sign}(\hat{\beta}_i),$$

where $\hat{\beta}_i$ is the i 'th entry of $\hat{\beta}_{MLS}$ defined above in (2.11) and $(x)_+ = \max\{0, x\}$. This expression is called the *soft threshold function*.

The ability of the LASSO and more specifically the ℓ_1 penalty to induce sparsity, allows us to make the connection with compressive sensing algorithms. Specifically the LASSO is connected to basis pursuit as follows

- If $\hat{\beta}_{lasso}$ is a minimizer of the LASSO in (2.12), then there exists $\eta = \eta_x \geq 0$ such that $\hat{\beta}_{lasso}$ is a minimizer of the quadratically constrained basis pursuit in (2.9).
- If $\hat{\beta}_{basis}$ is a unique minimizer of the quadratically constrained basis pursuit in (2.9), then there exists $\lambda = \lambda_x \geq 0$ such that $\hat{\beta}$ is a unique minimizer of the LASSO in (2.12).

This connection between LASSO and basis pursuit allows us to use results found in the context of compressive sensing to find conditions for the structure of X to evaluate $\hat{\beta}_{lasso}$ as an estimator for the true underlying β [10]. A similar connection will be found, by comparing a specific autoencoder model to a compressive sensing algorithm, that we are able to infer properties on the quality of the reconstruction of a dataset.

However, we must first introduce feature extraction in the form of PCA.

2.2 Feature Extraction

The underlying thread in the previous section shows that regression methods encounter problems when the design matrix is not of full rank, as a result of either super-collinearity or high-dimensionality. While ridge regression and LASSO regression seek to remedy these problems, we can also aim to find a lower-dimensional representation of the data X that aims to keep some reconstruction error as small as possible. This lower dimensional representation Z of X either completely eliminates any dependency in the columns of X or simply reduces the computational complexity of further analysis at the cost of minimal information loss in X .

While the specific need for dimensionality reduction tools follows naturally from the regression context sketched above, dimensionality reduction, and specifically feature extraction, has a much broader implementation, notably in classification of labeled datasets and in the interpretability of components in datasets.

2.2.1 Principal Component Analysis

To introduce the notion of feature extraction, we start with a popular and classical technique called *Principal Component Analysis* (PCA), that is used to find lower-dimensional representations of data [15].

The idea of PCA is to reduce the dimensionality of a dataset consisting of many dependencies between components, while retaining as much as possible of the variation present in the dataset. This is achieved by initially transforming the *observation matrix* $X \in \mathbb{R}^{n \times p}$, with rows consisting of the observations in our dataset as defined in Section 2.1.2, with the additional constraint that each column of X has empirical mean 0. Then, we can linearly transform the shifted X to a representation Z_x with a new set of components called the *principal components* (PCs), which are uncorrelated and ordered so that the first few components retain most of the variation present in all of the original components.

To achieve the goal of PCA, we wish to initially find a vector $v_1 \in \mathbb{R}^p$ that maximizes $\mathbb{V}(Xv)$, i.e.

$$v_1 = \operatorname{argmax} \mathbb{V}(Xv) = \operatorname{argmax} v^T X^T X v. \quad (2.13)$$

The v_1 that indeed maximizes $\mathbb{V}(Xv_1)$ is called the 1st *loading vector* and the resulting transformation $z_1 = Xv_1$ is called the 1st *principal component*.

Subsequently, we want to find another vector v_2 that also maximizes $\mathbb{V}(Xv)$ while simultaneously having the property that Xv_2 is uncorrelated to Xv_1 , i.e.

$$\operatorname{cov}(Xv_1, Xv_2) = v_1^T X^T X v_2 = v_2^T X^T X v_1 = 0.$$

The v_2 that satisfies these properties is the 2nd loading vector and $z_2 = Xv_2$ is again the 2nd principal component. If we continue this k -times we end up with $Xv_1, \dots, Xv_k$, the first k principal components, where for each $i, j \in \{0, \dots, k\}$ and $i \neq j$ we have $\operatorname{cov}(Xv_i, Xv_j) = 0$.

It is easy to see that without any further constraints, no finite v can maximize $\mathbb{V}(Xv)$. Hence, we constrain each v_i such that $v_i^T v_i = 1$. It turns out, with this added constraint, the maximal value that $\mathbb{V}(Xv)$ in (2.13) can obtain is λ_1 , the largest eigenvalue of $X^T X$ and consequently, the 1st loading vector v_1 such that

$$\mathbb{V}(Xv_1) = \lambda_1,$$

is also the eigenvector corresponding to λ_1 . It then also follows that each i 'th loading vector v_i is the eigenvector corresponding to λ_i , the i -th largest eigenvalue of $X^T X$. The

loading vectors therefore span an orthogonal basis where each coordinate is ordered by the amount of variance explained in the data X . The principal components, therefore are the data projected onto this new basis.

We can alternatively define PCA as expressed above in matrix notation:

Definition 2.4 (Principal Component Analysis). Principal Component Analysis can be defined as the orthogonal linear transform

$$Z_x = XV_x, \quad (2.14)$$

where $X \in \mathbb{R}^{n \times p}$ is the column-wise centered matrix of observations, $Z_x \in \mathbb{R}^{n \times p}$ is a orthogonal matrix and the columns of $V_x \in \mathbb{R}^{p \times p}$ are the right singular vectors of X obtained via the singular value decomposition

$$X = U_x D_x V_x^T.$$

The columns of Z_x are called the *principal components* and the columns of V_x are called the *loading vectors* or loadings.

We will from now on refer to this definition of PCA as "standard PCA".

The principal components in Z_x successively inherit the maximum possible variance from X . Therefore when we want to reduce the dimensionality of X , by keeping only the first $k < p$ principal components, we simply apply the truncated transformation

$$Z_{x,k} V_{x,k}^T = X_k^\sharp,$$

to find a reproduction $X_k^\sharp$ of X . Here the columns of $Z_{x,k}$ are the first k principal components and first k columns $V_{x,k}$ are the first k loading vectors. Using this transformation, we minimize the information lost by maximizing the variances of the k principal components that are kept [19].

Two important properties of the principal components and the loading vectors are that the principal components are uncorrelated and the loading vectors are orthogonal, i.e. $V_x^T X^T X V_x$ is diagonal and $V_x^T V_x = \mathbf{I}_p$. The uncorrelatedness of $V_{x,k}$ allows us to calculate the total variance explained by $V_{x,k}$ simply by

$$\mathbb{V}(XV_{x,k}) = \text{tr}(Z_{x,k}^T Z_{x,k}).$$

The connection between PCA and the eigenvectors of $X^T X$ allows us to derive a similar alternative expression for Z_x using the SVD of X :

$$Z_x = U_x D_x V_x^T V_x = U_x D_x,$$

where the SVD of $X = U_x D_x V_x^T$ is as defined above.

This connection is further explored in the fact that $V_{x,k}$ is a solution to the minimization problem

$$\arg \min_{V_k: V_k^T V_k = \mathbf{I}_k} \|X - V_k V_k^T X\|_F^2, \quad (2.15)$$

where V_k is a matrix of rank k and $\|\cdot\|_F$ is the Frobenius norm. This is a consequence of the Eckart–Young–Mirsky theorem [19] (Appendix A).

To see this, we note three identities that are easily found by computation:

$$V_{x,k}^T V_x D_x = [\mathbf{I}_k \quad \mathbf{0}_{k,p-k}] \quad (2.16)$$

$$V_{x,k} [\mathbf{I}_k \quad \mathbf{0}_{k,p-k}] = V_x \begin{bmatrix} \mathbf{I}_k & \mathbf{0}_{k,p-k} \\ \mathbf{0}_{n-k,k} & \mathbf{0}_{n-k,p-k} \end{bmatrix} \quad (2.17)$$

$$\begin{bmatrix} \mathbf{I}_k & \mathbf{0}_{k,p-k} \\ \mathbf{0}_{n-k,k} & \mathbf{0}_{n-k,p-k} \end{bmatrix} D_x = \begin{bmatrix} D_{x,k} & \mathbf{0}_{k,p-k} \\ \mathbf{0}_{n-k,k} & \mathbf{0}_{n-k,p-k} \end{bmatrix} =: D_{x,k}^* \quad (2.18)$$

for $D_{x,k}$ the diagonal matrix consisting of the k largest eigenvalues of X , $V_{x,k}$ the first k columns of V_x and $U_{x,k}$ the first k rows of U_x .

If we then set $V_k = V_{x,k}$ we can apply (2.16) - (2.18) to find

$$\begin{aligned} V_k V_k^T X &= V_{x,k} V_{x,k}^T U_x D_x V_x^T \\ &= V_{x,k} V_{x,k}^T V_x D_x U_x^T \\ &= V_x D_{x,k} U_x^T \\ &= U_x D_{x,k} V_x^T \end{aligned}$$

which leads us to conclude that $V_k V_k^T X$ is the k -thin SVD of X (Appendix A) and therefore the best rank k approximation of X in the Frobenius norm according to the Eckart–Young–Mirsky theorem.

We notice that the PCA minimization problem in (2.15) resembles finding an h that minimizes the AMSE in (2.4). We see that if we choose the smoother in the regression problem as

$$\hat{r}_{h,n}(X) = V_h V_h^T X,$$

where the discrete smoothing parameter h is the rank of V_h , i.e that $\hat{r}_{h,n}(X)$ reconstructs X from the first h loading vectors. Then the V_h that minimizes $R(h)$ in (2.4) is V_h that minimizes the problem above in (2.15). The key difference is that in (2.4) we want to find an h that minimizes the AMSE for a specific smoother and in (2.15) we fix h and find a different parameter of the smoother, V_k in this case, that minimizes a loss function that resembles average mean square error. However, this resemblance between problems leads to the use of regression methods covered in Section 2.1.2 to find an alternative expression for PCA and augment PCA to find sparse loading vectors.

2.2.2 Ridge PCA

Our motivation for finding an expression for PCA alternative to (2.15) lies in the fact that for any $k < p$, the V_k that minimizes (2.14) is not unique. We see that if we choose some orthogonal matrix $T \neq \mathbf{I}_k$ such that $V'_k = V_{x,k}T$, then not only $V_{x,k}$, the first k loading vectors of PCA, solves (2.15), but also V'_k .

Furthermore, in a high-dimensional setting, when $p \gg n$, even when V_k is of full rank, i.e. $k = p$, we cannot find a unique V_k that minimizes (2.15).

To remedy this problem, we can apply techniques from regression in the form of an adjusted ridge regression problem. To recover exactly V_x , all p ordered eigenvectors of $X^T X$, we find that rank p matrix $\hat{V}$ that minimizes

$$\arg \min_{V, W: V^T V = \mathbf{I}_p} \|X - WV^T X\|_F^2 + \lambda \|V\|_F^2. \quad (2.19)$$

This $\hat{V}$ has the property that for each $i = \{0, \dots, p\}$, $\hat{V}_{(:,i)} \propto V_{x(:,i)}$, i.e. each column of $\hat{V}$ is proportional to the corresponding column of V_x . Therefore, normalizing $\hat{V}$ column-wise we can recover exactly V_x and not just up to some orthogonal rotation as in (2.15) [16].

Actually, (2.19) holds for any rank $k \leq p$ matrix $\hat{V}_k$, via the following theorem

Theorem 2.5 (Theorem 3 of [16]). *The rank k matrix $\hat{V}_k$ that minimizes*

$$\arg \min_{V, W: V^T V = \mathbf{I}_k, \text{rank}(V)=k} \|X - WV^T X\|_F^2 + \lambda \|V\|_F^2, \quad (2.20)$$

has the property that, for each $i \in \{1, \dots, k\}$, $\hat{V}_{k(:,i)} \propto V_{x(:,i)}$.

This means that we can always recover the i 'th loading vector as derived in standard PCA from the i 'th column of $\hat{V}_k$ for $i \in \{1, \dots, k\}$, by solving (2.20).

2.2.3 Sparse PCA

The next logical step is to modify the idea of PCA using the LASSO in (2.12). We can modify the minimization objective (2.20) by adding a ℓ_1 penalty such that the minimization objective becomes for each $k < p$

$$\arg \min_{V, W: V^T V = \mathbf{I}_k, \text{rank}(V)=k} \|X - WV^T X\|_F^2 + \lambda \|V\|_F^2 + \sum_{i=1}^k \lambda_{1,k} \|V_{(:,i)}\|_1. \quad (2.21)$$

Note that importantly $\lambda > 0$ in (2.20) is scalar and $\lambda_1 = [\lambda_1, 1, \dots, \lambda_1, k]^T$ above is a vector such that we can individually penalize different loadings [16].

With the addition of the ℓ_1 penalty, the new loading vectors, denoted by V^l , enforce sparsity such that some components in the loading vectors are set to zero, essentially

selecting which entries of the observations in X are relevant in calculating the new adjusted principal components. Note that, critically, sparse loading vectors do not in turn produce sparse principal components.

Furthermore, the adjusted loading vectors are forced to retain their orthogonality via the orthogonality constraint in the minimization problem, however the adjusted principal components Z^l , which are recovered from $Z^l = XV^l$ are not automatically uncorrelated anymore. Specifically only the loading vectors of ordinary PCA can be both orthogonal and uncorrelated [17].

The loss of uncorrelated principal components also requires a new derivation of the explained total variance. Now, this is calculated to be

$$\mathbb{V}(Z^l) = \sum_{i=1}^k R_{i,i}^2,$$

where $R_{i,i}$ are the diagonal entries of the upper triangular matrix taken from the QR decomposition of Z^l [16].

The desire to add variable selection in PCA arises in the interpretation of the principal components. By design, principal components are linear combinations of all p variables and the loadings are often non-zero. These two properties often make it difficult to interpret the derived principal components. Variable selection is then added, due to its aid in interpretability, to combat these issues [16].

2.3 Neural Networks

With the goal of expanding on the concept of PCA, we first want to introduce the broad framework of neural networks that we wish to apply to the PCA problem.

Let us now again consider the same observation matrix X from our experiment defined in Section 2.1.2. To keep consistent with the notation used in the context of regression as well as to simplify the notation going forward, we choose to use X^T as the input of our neural networks going forward.

We can define a *multilayer neural network* by an activation function and network architecture. The *activation function* is defined as $\sigma : \mathbb{R}^k \rightarrow \mathbb{R}^k$ with corresponding *shifted activation function* σ_b , defined as

$$\sigma_b \begin{pmatrix} x_1 \\ \vdots \\ x_k \end{pmatrix} = \begin{pmatrix} \sigma(x_1 - b_1) \\ \vdots \\ \sigma(x_k - b_k) \end{pmatrix},$$

where $b = [b_1, \dots, b_k]^T \in \mathbb{R}^k$ is called the *shift function*. The *network architecture* $(L, \mathbf{p})$ consists of a positive integer L called the *number of hidden layers* or *depth* as well as a *width vector* $\mathbf{p} = [p_0, \dots, p_{L+1}] \in \mathbb{R}^{L+2}$. The definition then follows:

Definition 2.6 (Multilayer Neural Network). A *multilayer neural network* with *network architecture* $(L, \mathbf{p})$ is defined as any function of the form

$$\begin{aligned} f : \mathbb{R}^{p_0} \rightarrow \mathbb{R}^{p_{L+1}}, \quad x \mapsto f(x) &= \sigma_{b_{L+1}} \circ W_{L+1} \cdot \sigma_{b_L} \circ W_L \cdot \dots \cdot \sigma_{b_2} \circ W_2 \cdot \sigma_{b_1} \circ W_1 x \\ &:= \sigma_{b_{L+1}} W_{L+1} \sigma_{b_L} W_L \dots \sigma_{b_2} W_2 \sigma_{b_1} W_1 x, \end{aligned}$$

where $f(x)$ is called a *network function* and each $W_i \in \mathbb{R}^{p_{i-1} \times p_i}$ and $b_i \in \mathbb{R}^{p_i}$ are respectively called the *weight matrices* and shift vectors or *biases*.

The network function of a neural network is therefore built by alternating matrix multiplications with the action of applying a (possibly nonlinear) activation function σ [18]³.

An appealing property of neural networks, relevant to the different regression problems we have stated so far, comes in the form of the universal approximation theorem. This theorem states that a neural network with one hidden layer, i.e. $L = 1$ can approximate any continuous function f arbitrarily well with respect to the uniform norm, provided there are enough hidden units, i.e. that p_1 , the width of $\sigma_{b_1} W_1 x$ is sufficiently large.

Because regression problems are essentially function estimation problems, the universal approximation theorem makes regression problems an appealing application of neural networks. Importantly, this theorem is only an existence theorem and does not guarantee that a specific network architecture will produce a good approximation. Finding and choosing an appropriate architecture is therefore crucial when using neural networks [18].

2.3.1 PCA as Neural Networks

As we have seen in Sections 2.2.2 and 2.2.3, PCA can also be defined as a type of regression problem. Therefore it is natural to assume that there are neural networks with specific architectures that can solve the regression problem connected to PCA.

Most existing methods that have been proposed to perform PCA via the use of neural networks, revolve around a specific iterative learning method called *Hebbian learning*.

Assuming the observations come from the observation matrix X , each method involves a neural network with architecture $(0, \{p, k\})$ for $k \leq p$ and network function $Y = f(X) = \sigma_b W X^T$ where $b = (0, \dots, 0)^T$. Then, $W(t)$ the weight matrix at step t is iteratively determined by some variation of the rule

$$\nabla_{W(t)} = \eta X(t) Y(t),$$

where ∇_W is the gradient of W , as defined below in Section 2.3.3, $X(t)$ is computed as

$$X^T(t) = X^T(t-1) - Y(t-1)W(t-1),$$

³We choose to use different indices for W_i and σ_{b_i} than is used in [18]. We wish to use the index to indicate that we are in layer i *after* we have applied W_i and σ_{b_i} to our input.

where $Y(t) = f(X(t))$ and $\eta > 0$ is some chosen hyperparameter [23].

An important method to highlight is when one chooses σ_b as the *ReLU activation function*, $\sigma_b(w_{i,j}x_{i,j}) = \max\{0, w_{i,j}x_{i,j}\}$, the update rule as

$$\nabla_{W(t)} = \eta[X(t) - Y(t)W(t-1)]Y(t)$$

and $Y \in \mathbb{R}^{n \times k}$ for specifically $k < p$. Then it has been shown that this neural network successfully learns exactly the first k principal components such that $Y = Z_{x,k}$ [24].

Despite the seemingly successful existing implementations of neural networks to perform PCA, we want to focus on the connection between PCA and a specific type of neural network called an autoencoder.

2.3.2 Autoencoders

A specific class of neural network architectures called *autoencoders* are neural networks tasked with finding efficient representations (encodings), often in lower dimensions, of the observation matrix X such that they can be reproduced (decoded) from these more efficient representations. Autoencoders are defined through the following components.

We can divide the network architecture into layers for respectively, the *encoder* and *decoder* as

$$(L, \mathbf{p}) = (L, \mathbf{p}_E, \mathbf{p}_D) = (L, \{p_0, \dots, p_e\}, \{p_{e+1}, \dots, p_{L+1}\}),$$

for some $e \leq L$ such that the layers of the encoder have widths $\mathbf{p}_E$, the layers of the decoder have widths $\mathbf{p}_D$. We additionally assume that $p_0 = p_{L+1}$. We will also mostly assume that $p_e < p_0$, in this case the autoencoder is called *undercomplete*. We can then define the decoded space $\mathcal{X} \in \mathbb{R}^{p_0} = \mathbb{R}^{p_{L+1}}$ and encoded space $\mathcal{Z} \in \mathbb{R}^{p_e}$. The encoded space is then accompanied by a parameterized encoder function,

$$E_{\theta_e} : \mathcal{X} \rightarrow \mathcal{Z}, \quad x \mapsto \sigma_{b_e} W_e \sigma_{b_{e-1}} \dots W_2 \sigma_{b_1} W_1 x = z$$

where $\theta_e = \{W_0, \dots, W_e, b_1, \dots, b_e\}$. As well as the decoded space being accompanied by a parameterized decoder function

$$D_{\theta_d} : \mathcal{Z} \rightarrow \mathcal{X}, \quad z \mapsto \sigma_{b_{L+1}} W_{L+1} \sigma_{b_L} \dots W_{e+2} \sigma_{b_{e+1}} W_{e+1} z = x^\sharp.$$

where $\theta_d = \{W_{e+1}, \dots, W_{L+1}, b_{e+1}, \dots, b_{L+1}\}$.

The network function $f(x)$ of an autoencoder can then instead be expressed by the encoder and decoder functions as

$$f(x) = D_{\theta_d}(E_{\theta_e}(x)) := D(E(x)) = x^\sharp,$$

where $x^\sharp$ is the reconstruction of x via the autoencoder defined by the encoder E and decoder D ⁴.

⁴We choose to drop θ_e and θ_d in the notation as the dependency on these parameters is clear

We then wish to train the autoencoder by finding values for θ_e and θ_d that minimize some loss function. The quality of an autoencoder is judged by a task defined by a reference probability distribution μ_{ref} and reconstruction measure $d : \mathcal{X} \times \mathcal{X} \rightarrow [0, \infty]$. According to this task, we can define the loss function of an autoencoder as

$$\mathcal{L}(\theta_e, \theta_d) := \mathbb{E}_{x \sim \mu_{ref}} (d[x, D(E(x))]). \quad (2.22)$$

Training the autoencoder then comes down to finding optimal values for θ_e and θ_d through the optimization problem [25]

$$\arg \min_{\theta_e, \theta_d} \mathcal{L}(\theta_e, \theta_d).$$

Luckily, autoencoders and neural networks in general benefit from efficient techniques that can be exploited to solve the resulting optimization problem that accompanies each network architecture.

2.3.3 Optimization

Specific techniques have been developed to optimize the loss function associated with a neural-network. These techniques cleverly incorporate the structure of neural networks to efficiently find the desired weights that minimize the loss function of interest.

The most basic technique used is that of *gradient descent*. This is a step-wise method where the desired parameters $\theta = \{W_1, \dots, W_{L+1}, b_1, \dots, b_{L+1}\}$ of $\mathcal{L}$, the loss function associated to a neural network consisting of L layers, that need to be optimized, are updated at each step t according to the gradient $g_t(\theta)$ and some learning rate η [21].

Definition 2.7 (Gradient). The *gradient* of θ is defined as the vector of partial derivatives of the associated loss function $\mathcal{L}$ in terms of θ at step t , i.e

$$\nabla_{\theta} \mathcal{L}_t(\theta) := \left(\frac{\partial \mathcal{L}_t}{\partial \theta} \right)^T = \left[\frac{\partial \mathcal{L}_t}{\partial W_1}, \dots, \frac{\partial \mathcal{L}_t}{\partial W_{L+1}}, \frac{\partial \mathcal{L}_t}{\partial b_1}, \dots, \frac{\partial \mathcal{L}_t}{\partial b_{L+1}} \right]^T$$

where $\mathcal{L}_t(\theta)$ is the realization of the loss function at step t .

The learning rate η is a hyper-parameter that determines by how much the parameters are updated at each step. The gradient descent update rule at time step t then becomes

$$\theta_t = \theta_{t-1} - \eta \nabla_{\theta} \mathcal{L}_t(\theta). \quad (2.23)$$

The maximal value of t can then either be fixed or until convergence of θ_t occurs.

Furthermore, an initial choice of θ at $t = 0$ needs to be chosen beforehand such that the realization $\mathcal{L}_0(\theta)$ can be computed. This is called the *initialization* of θ .

The structure of a neural-network allows for efficient computation of the gradient $\nabla_{\theta} \mathcal{L}_t(\theta)$. This is done by a technique called *back-propagation*.

We can compute the derivative of the loss function for a neural network of depth L for some input x , in terms of weighted input $x_l^w = W_l \sigma_{l-1} W_{l-1} \dots \sigma_1 W_1 x$ and the output $x_l^* = \sigma_l x_l^w$ of layer l , for each hidden layer $l \leq L$. This can be expressed as

$$\frac{\partial \mathcal{L}}{\partial x_L^*} \cdot \frac{\partial x_L^w}{\partial x_{L-1}^*} \cdot \frac{\partial x_{L-1}^*}{\partial x_{L-1}^w} \cdot \frac{\partial x_{L-1}^w}{\partial x_{L-2}^*} \cdot \dots \cdot \frac{\partial x_3^*}{\partial x_2^w} \cdot \frac{\partial x_2^w}{\partial x_1^*} \cdot \frac{\partial x_1^*}{\partial x_1^w} \cdot \frac{\partial x_1^w}{\partial x}. \quad (2.24)$$

We can then rewrite (2.24) in terms of W_l the weight matrices and $\sigma'_l := \sigma'_l(x_l^w)$ the derivatives of the activation functions evaluated at x_l^w as follows

$$\frac{\partial \mathcal{L}}{\partial x_L^*} \odot \sigma'_L \cdot W_L \odot \sigma'_{L-1} \cdot W_{L-1} \odot \dots \odot \sigma'_1 \cdot W_1, \quad (2.25)$$

here the operator $\odot$ denotes the Hadamard product. This follows from the fact that each $\frac{\partial x_l^w}{\partial x_{l-1}^*} = W_l$. The gradient $\nabla_x \mathcal{L}$ is then the transpose of the derivative in (2.25) above:

$$\nabla_x \mathcal{L} = W_1^T \cdot \sigma'_1 \odot \dots \odot W_L^T \cdot \sigma'_L \odot \nabla_{x_L^*} \mathcal{L}(\theta).$$

If we then take the gradient of the output at layer $l < L$

$$\nabla_{x_l^*} \mathcal{L} = \sigma'_l \odot W_{l+1}^T \cdot \sigma'_{l+1} \odot \dots \odot W_L^T \cdot \sigma'_L \odot \nabla_{x_L^*} \mathcal{L}(\theta),$$

we are able to subsequently express the gradient of the weights in layer l , W_l , only in terms of δ_l and x_{l-1}^* as follows

$$\nabla_{W_l} \mathcal{L} = \delta_l (x_{l-1}^*)^T.$$

The gradients can then be computed recursively from $\nabla_{W_l} \mathcal{L}$ to $\nabla_{W_1} \mathcal{L}$ as δ_l can be computed recursively as

$$\delta_{l-1} = \sigma'_{l-1} \odot W_l^T \cdot \delta_l$$

and $\delta_L = \sigma'_L \odot \nabla_{x_L^*} \mathcal{L}$. The benefit of computing the gradients in this order and using these formulations is that one does not have to repeat duplicate multiplications as well as one only needs to compute multiplications of vectors by matrices instead of matrix-matrix multiplication.

We also now have a different formulation of our gradient descent update rule for the weights in layer l at step t in terms of δ_l and x_{l-1}^* ,

$$W_{l,t} = W_{l-1,t} - \eta \delta_l x_{l-1}^*.$$

We can also similarly express the update rule at step t of the biases b_l in layer l in terms of δ_l as

$$b_{l,t} = b_{l-1,t} - \eta \delta_l.$$

A drawback of using gradient descent to find the optimal parameters θ associated with a neural network arises in the fact that gradient descent often finds values for θ such that only a local minimum is found for $\mathcal{L}(\theta)$. To combat this, *stochastic gradient descent* was developed [21]. The key difference is that gradient descent is computed according to the entire input X . Stochastic gradient descent instead introduces variance in the computed gradients by performing gradient descent on a different random observation $x \in X$ for every step t . The introduced variance in the gradients attempt to escape any possible local minima of $L(\theta)$. A compromise between gradient descent and "true" stochastic gradient descent is that for each t a different random subset (mini-batch) $X^t \subset X$ is used as input.

A further iteration of gradient descent, as well as the main optimization technique used going forward is the *Adam optimizer* [22]. Adam introduces additional parameters that are computed and used in an adjusted update rule. The hyperparameters that need to be chosen are η , the learning rate, $\beta_1, \beta_2 \in [0, 1)$, exponential rates of decay for m_t and v_t below and ϵ , a bias term used in the update rule. Additionally the new parameters $m_t, v_t, \hat{m}_t, \hat{v}_t$ are defined at step t as

$$\begin{aligned} m_t &= \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot \nabla_{\theta} \mathcal{L}_1 && \text{(Biased first moment estimate)} \\ v_t &= \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot \nabla_{\theta} \mathcal{L}_t \odot \nabla_{\theta} \mathcal{L}_t && \text{(Biased raw second moment estimate)} \\ \hat{m}_t &= \frac{m_t}{1 - \beta_1^t} && \text{(Biased-corrected first moment estimate)} \\ \hat{v}_t &= \frac{v_t}{1 - \beta_2^t} && \text{(Biased-corrected raw second moment estimate)}. \end{aligned}$$

The initialization for m_0, v_0 are taken to be 0. The Adam update rule at step t then becomes

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}. \quad (2.26)$$

3 Results

Equipped with a better understanding of feature extraction in the form of PCA as well as a broad framework in the form of neural networks and more specifically autoencoders, we now wish to apply the concept of PCA to the framework of autoencoders.

Our goal is twofold, firstly to be able to replicate standard PCA using the architecture of autoencoders as well as the efficient optimization techniques made possible through the use of autoencoders.

Secondly, if we are able to replicate standard PCA through the use of autoencoders, can we use the generality of autoencoder architectures to find adjusted forms of PCA that display additional properties, where standard PCA would otherwise remain too restrictive.

3.1 PCA as Autoencoders

The initial step in formulating an autoencoder as PCA is to make specific choices for μ_{ref} and d in (2.22). If we set μ_{ref} the reference distribution of X to be the empirical distribution of the dataset $X \in \mathbb{R}^{n \times p}$ and choose d to be the ℓ_2 norm, we see that the autoencoder loss function $\mathcal{L}(\theta_e, \theta_d)$ (2.22) becomes

$$\mathcal{L}(\theta_e, \theta_d) = \frac{1}{n} \sum_{i=1}^n \|x_i^T - D(E(x_i^T))\|_2^2 = \frac{1}{n} \|X^T - D(E(X^T))\|_F^2,$$

where we again mention that the encoder E and decoder D respectively depend on parameters θ_e and θ_d . We will furthermore also assume that μ_{ref} and d are chosen as such in the loss functions of all further autoencoders we construct.

As a result of this formulation, finding the θ_e and θ_d that minimize $\mathcal{L}(\theta_e, \theta_d)$ is equivalent to solving

$$\arg \min_{\theta_e, \theta_d} \|X^T - D(E(X^T))\|_F^2. \quad (3.1)$$

We immediately see that this autoencoder optimization problem loosely resembles the minimization problem of PCA in (2.15):

$$\arg \min_{V_k: V_k^T V_k = \mathbf{I}_k} \|X - V_k V_k^T X\|_F^2.$$

It therefore seems that one might be able to choose a network architecture for some autoencoder with these specific choices for μ_{ref} and d that can replicate the results of PCA.

3.1.1 Linear Autoencoder

Due to the linearity of PCA, a logical step in replicating PCA in terms of the autoencoder framework would be to inspect one of the simpler classes of autoencoders, called *linear autoencoders*.

This class of autoencoders is classified by choosing all activation functions $\sigma_{b_0}, \dots, \sigma_{b_{L+1}}$ as linear. This allows us to simplify both the encoder and decoder to the form $E(x) = W_E x + b_E$ and $D(z) = W_D z + b_D$. We will refer to W_E, b_E and W_D, b_D as the *encoder weight and bias* and *decoder weight and bias* respectively. With these choices for our encoder and decoder, we are left with an autoencoder with architecture $(L, \mathbf{p}_e, \mathbf{p}_d) = (1, \{p, k\}, \{p\})$ our optimization task now becomes

$$\arg \min_{W_E, W_D, b_E, b_D} \|X^T - W_D(W_E X^T + b_E \mathbf{1}_n^T) + b_D \mathbf{1}_n^T\|_F^2, \quad (3.2)$$

where $\mathbf{1}_n$ is a column vector of ones of length n .

If we fix $W_E = W_D^T = W$, the optima for b_E, b_D the encoder and decoder biases in (3.2), are such that we can restate (3.2) as

$$\arg \min_{W: W^T W = \mathbf{I}_k} \|X_0^T - W W^T X_0^T\|_F^2, \quad (3.3)$$

where

$$X_0^T = X^T \left(\mathbf{I}_n - \frac{\mathbf{1}_n \mathbf{1}_n^T}{n} \right) = X^T - \frac{X \mathbf{1}_n}{n} \mathbf{1}_n^T = X^T - \mu_X \mathbf{1}_n^T, \quad (3.4)$$

and μ_X is defined for each $j \in \{1, \dots, p\}$ such that

$$\mu_{X_j} = \frac{1}{n} \sum_{i=1}^n X_{(i,j)},$$

i.e. the j 'th entry of μ_X is the empirical mean of the j 'th entry of all observations in the dataset X .

Therefore, X_0 is the column-wise centered version of X and as a consequence, the linear autoencoder framework automatically pre-processes X according to the requirements of PCA as part of the optimization. We also see that we have now formulated the PCA minimization problem in (2.15) in terms of the autoencoder framework, on the condition that we are able to enforce the constraint of orthogonality on W .

The orthogonality of W in (3.3) follows from another automatic property that arises in the optimization task (3.2) in the form of the optimal encoder weight $\widehat{W}_E$ of (3.2).

If we do not assume that $W_E = W_D^T$, we can state the optimal $\widehat{W}_E$ in terms of W_D , for any value of W_D , as

$$\widehat{W}_E = (W_D^T W_D)^{-1} W_D.$$

However, if we now add the assumption that $W_E = W_D^T$ as in (3.3), we can additionally state $\widehat{W}_E$ in terms of W_E , for any value of W_E as $\widehat{W}_E = (W_E)^{-1}$. Then, by choosing $W_E = \widehat{W}_E$ we see that

$$\widehat{W}_E^T \widehat{W}_E = W_E^{-1} W_E = \mathbf{I}_k.$$

Therefore, the optimal W in (3.3) will be orthogonal regardless of the imposed hard constraint and as a result we can state (3.3) as the unconstrained problem

$$\arg \min_W \|X_0 - W W^T X_0\|_F^2. \quad (3.5)$$

This then implies that we now have an unconstrained autoencoder that can solve the PCA minimization problem.

Due to the chosen dimension of the encoded space $\mathcal{Z} \in \mathbb{R}^k$, the weight matrix W has rank k and therefore, as mentioned in Section 2.2.2, there exists an orthogonal matrix T such that $\widehat{W} = V_{x,k} T$, for $\widehat{W}$ the solution to (3.5). Therefore we are able to reproduce, up to orthogonal rotations, the first k loading vectors of X obtained via standard PCA (after pre-processing X to be centered). Similarly, the encoder then also replicates the first k principal components $Z_{x,k}$ up to some orthogonal rotation¹:

$$E(X) = \widehat{W}^T X^T = (X V_{x,k} T)^T = (Z_{x,k} T)^T.$$

As a result of $\widehat{W}$ in (3.5) being a rotated version of $V_{x,k}$, the columns of $\widehat{W}$ are not necessarily sorted in terms of variance. Apart from explicitly calculating the variance of each column of $\widehat{W}$, we are also not able to tell how the columns of $\widehat{W}$ are ordered. Therefore, we cannot extract the first $l < k$ loading vectors from $\widehat{W}_E$ simply by choosing the first l columns of $\widehat{W}$. To do this we would either have to calculate the variances of each column or retrain a new encoder with hidden layer width l .

This shortcoming of the autoencoder formulated in (3.5) requires us to find a different formulation of PCA that we can express in the framework of autoencoders that can indeed find some ordered W_E .

¹If we release any constraints on W_E and W_D and assume the autoencoder to be as in (3.2), it has been empirically shown in [19] that we can recover the first k loading vectors from the left singular vectors of W_D with the hidden layer of the autoencoder having width k , i.e for $W_D = U_D D_D V_D^T$, the SVD of W_D we have $V_{x,k} = W_D$, where $V_{x,k}$ are the first k loading vectors of X as usual. [19]

3.1.2 Ridge and Sparse Linear Autoencoders

We turn to the alternative formulation of PCA as a ridge regression type problem in (2.20) where the optimal $\widehat{V}_k$ that minimizes

$$\arg \min_{V, W: V^T V = \mathbf{I}_k, \text{rank}(V)=k} \|X - WV^T X\|_F^2 + \lambda \|V\|_F^2, \quad (3.6)$$

has the property that, for each $i \in \{1, \dots, k\}$, $\widehat{V}_{k(\cdot, i)} \propto V_{x(\cdot, i)}$ and specifically $V_{x(\cdot, i)} = \widehat{V}_{k(\cdot, i)} / \|\widehat{V}_{k(\cdot, i)}\|_2$.

We immediately see that this formulation and resulting property allows us, to use a minimization problem to find, after normalization, exactly $V_{x,k}$ from $\widehat{V}_k$, where also $\widehat{V}_k$ has columns ordered by variance.

To replicate the ridge PCA formulation above in (2.20) as an autoencoder, we can consider the linear autoencoder in (3.2) with an adjusted loss function by adding an ℓ_2 penalty on W_E as follows

$$\arg \min_{W_E, W_D, b_E, b_D} \|X^T - W_D(W_E X^T + b_E \mathbf{1}_n) + b_D \mathbf{1}_n\|_F^2 + \lambda \|W_E\|_F^2, \quad (3.7)$$

for some chosen hyperparameter $\lambda > 0$.

Because the additional penalty does not depend on the biases b_E and b_D , the optimal values for b_E and b_D do not change compared to (3.2) in such that we can restate (3.7) in terms of the column-wise centered X_0 , as the optimization problem²

$$\arg \min_{W_E, W_D: W_E^T W_E = \mathbf{I}_k} \|X_0^T - W_D W_E X_0^T\|_F^2 + \lambda \|W_E\|_F^2. \quad (3.8)$$

This formulation of the ridge PCA problem into an autoencoder successfully allows us to replicate each of the k first **columns** of V_x from the **rows** of the optimal $\widehat{W}_E$ in (3.8) as

$$V_{x(\cdot, i)} = \frac{\widehat{W}_{E(i, \cdot)}}{\|\widehat{W}_{E(i, \cdot)}\|_2}.$$

However, this is only true if we can reformulate the hard constraint of $W_E^T W_E = \mathbf{I}_k$ in such a way that it can be implemented into the framework of autoencoders. Which is exactly what we will show below.

²Note that choosing the hidden layer to have width k implies that $\text{rank}(W_E) = k$ and therefore we do not have to impose a hard constraint on the rank of W_E .

Orthogonal Constraints of Weight Matrices

The context of the connection between autoencoders and PCA provides us with a need to be able to impose orthogonality on the encoder weights.

The straightforward way is to impose a soft orthogonality constraint by adding a Lagrange multiplier to the loss function of an autoencoder such that the optimization task takes the form

$$\arg \min_{W_E, W_D, b_E, b_D} \|X - D(E(X))\|_F^2 + \lambda \|W_*^T W_* - \mathbf{I}\|_F^2,$$

where W_* is any of the weight matrices in either the encoder or decoder and $\lambda > 0$ is a hyperparameter chosen to adjust how much non-orthogonality of W is penalized. Immediately, one notices that the addition of such a multiplier encourages W_* to be orthogonal, but certainly does not ensure it. If we wish to impose a hard constraint of the orthogonality of W_* , a different technique is needed.

Optimization with a hard orthogonality constraint is more generally known in literature as optimization on manifolds with orthogonality constraints [27]. This more general problem can be stated as solving

$$\arg \min_{M: M^T M = \mathbf{I}_p} \mathcal{F}(M), \quad (3.9)$$

where M is a $k \times p$ matrix and $\mathcal{F} : \mathbb{R}^{k \times p} \rightarrow \mathbb{R}$ a differentiable function. The solution to (3.9), lies in the set $\mathcal{S}_{k,p} := \{M \in \mathbb{R}^{k \times p} : M^T M = \mathbf{I}_p\}$ called the Stiefel manifold.

While different methods to solving (3.9) have been presented [27], the method used in the Python library `GeoTorch` [26] and consequently used in simulations here, is applied as follows.

We can express a matrix $B \in \mathcal{S}_{k,p}$ as $B = \exp(A)$, where $\exp$ is the map defined as

$$\exp : \mathfrak{s}_k \rightarrow \mathcal{S}_{k,p}, \quad A \mapsto \mathbf{I} + A + \frac{1}{2}A^2 + \dots + \frac{1}{n!}A^n + \dots$$

and $A \in \{M \in \mathbb{R}^{k \times k} : M - M^T = 0\} =: \mathfrak{s}_k$ is a skew-symmetric matrix. This equivalence then implies that solving (3.9) is equivalent to solving

$$\arg \min_{M: M=M^T} \mathcal{F}(\exp(M)). \quad (3.10)$$

Importantly, on the condition that the entries of $\exp(M)$ in (3.10) do not explode to infinity, (3.10) uniquely solves (3.9).

This reformulation of the optimization problem now allows us to use optimization tools from the neural network framework, as $\mathfrak{s}_k$ is isomorphic to a Euclidian vector space. Specifically, we can write $A \in \mathfrak{s}_k$ as

$$A = A_\Delta - A_\Delta^T, \quad (3.11)$$

where A_Δ is an upper triangular matrix with all diagonal entries being zero. We can then easily define a weight matrix W in an autoencoder to be of the form of A_Δ by fixing all entries on and below the diagonal of W to zero and adjusting the update rule in (2.23) or (2.26) to omit the partial derivatives of W associated to these entries in the gradient $\nabla_{\theta_e, \theta_d} \mathcal{L}(\theta_e, \theta_d)$.

We can then reformulate a hard orthogonality constraint of an encoder weight $W_l \in \theta_e$, in hidden layer $l < e$ of the encoder with width k as

$$\begin{aligned} & \arg \min_{\theta_e, \theta_d: W_l \in \mathcal{S}_{k,p}} \|X^T - D(\sigma_e W_e \dots \sigma_l W_l \sigma_{l-1} W_{l-1} \dots \sigma_1 W_1 X^T)\|_F^2 \\ &= \arg \min_{\theta_e, \theta_d: W \in \mathfrak{s}_k} \|X^T - D(\sigma_e W_e \dots \sigma_l \exp(W) \sigma_{l-1} W_{l-1} \dots \sigma_1 W_1 X^T)\|_F^2 \\ &= \arg \min_{\theta_e, \theta_d} \|X^T - D(\sigma_e W_e \dots \sigma_l \exp(W_\Delta) \exp(-W_\Delta^T) \sigma_{l-1} W_{l-1} \dots \sigma_1 W_1 X^T)\|_F^2, \end{aligned}$$

where $W_l = \exp(W)$ for $W \in \mathfrak{s}_k$ skew-symmetric and $W = W_\Delta - W_\Delta^T$ as in (3.11)³.

Ridge and Sparse PCA as autoencoders

Equipped with a tool to enforce orthogonality of weight matrices in autoencoders, we can now state the ridge PCA problem in (2.20) as an autoencoder with architecture $(L, \mathbf{p}) = (1, \{p, k\}, \{p\})$ for the centered input $X_0 \in \mathbb{R}^{n \times p}$ as defined previously, with the following optimization task

$$\begin{aligned} & \arg \min_{W_\Delta, W_D} \|X_0^T - W_D \exp(W_\Delta) \exp(-W_\Delta^T) X_0^T\|_F^2 \\ & + \lambda \|\exp(W_\Delta) \exp(-W_\Delta^T)\|_F^2, \end{aligned} \quad (3.12)$$

for $\lambda > 0$ a chosen hyperparameter and W_Δ as in (3.11) such that the encoder weight can be expressed as $W_E = \exp(W_\Delta - W_\Delta^T)$ and that $W_E^T W_E = \mathbf{I}_k$ holds. We can then recover the i 'th loading vector of standard PCA from the encoder weight $\widehat{W}_E$ that solves (3.12) as follows

$$V_{x(\cdot, i)} = \widehat{W}_{E^*(i, \cdot)} := \frac{\widehat{W}_{E(i, \cdot)}}{\|\widehat{W}_{E(i, \cdot)}\|_2}.$$

³We can also similarly formulate a hard orthogonality constraint for any decoder weight matrix.

Also, we are similarly able to recover the i 'th principal component from

$$Z_{x(\cdot, i)} = \left(\widehat{W}_{E^*(i, \cdot)} X_0^T \right)^T.$$

In contrast to the linear autoencoder in (3.5), we are now able to find $\widehat{W}_E^{k'}$, the optimal encoder weight for an autoencoder of the form (3.12) with hidden layer width $k'k$, directly from $\widehat{W}_E^k$, the optimal encoder weight for an autoencoder of the form (3.12) with hidden layer width $k > k'$, simply by truncating the last $k - k'$ rows of $\widehat{W}_E^k$ such that $\widehat{W}_E^{k'} = \widehat{W}_{E(1:l, \cdot)}^k$. This is made possible by the fact that the rows of the encoder $E(X)$ of (3.12) are uncorrelated as well as the rows of the optimal encoder weight $\widehat{W}_E$ being ordered in terms of variance. An encoder weight that satisfies these properties is called *nested* [19].

We can also formulate the sparse PCA problem in (2.21) as an autoencoder with architecture $(L, \mathbf{p}) = (1, \{p, k\}, \{p\})$ and centered input $X_0 \in \mathbb{R}^{n \times p}$, only now with the adjusted optimization task

$$\begin{aligned} \arg \min_{W_\Delta, W_D} & \|X_0^T - W_D \exp(W_\Delta) \exp(-W_\Delta^T) X_0^T\|_F^2 \\ & + \lambda \|\exp(W_\Delta) \exp(-W_\Delta^T)\|_F^2 + \sum_{i=1}^k \lambda_{1,i} \|W_{E(i, \cdot)}\|_1, \end{aligned} \quad (3.13)$$

where $\lambda_1 \in \mathbb{R}^k$.⁴ Here, similarly each row of W_E provides adjusted loading vectors and each row of $E(X_0) = W_E X_0^T$ provides an adjusted principal component, where also the rows of W_E are ordered in terms of variance. However, as the rows of $E(X_0)$ are now correlated, the first $k' < k$ rows of W_E are not able to minimize an autoencoder of the form (3.13) with a hidden layer width of k' . Therefore, just as with the linear autoencoder in (3.5), we would then have to retrain our autoencoder every time we wish to adjust the width of the hidden layer.

3.1.3 Numerical Illustration

As mentioned in Section 2.3, a common problem with neural networks is that while certain properties can be shown for optimal values of the associated optimization task, it is not guaranteed that during training we are able to find the optimal values for which these properties hold. This is due to the fact that unsuitable optimization methods often get stuck in local minima without reaching the true minimum of the optimization task.

⁴We denote $W_E = \exp(W_\Delta - W_\Delta^T)$ as such in the ℓ_1 penalty, to make clear that we penalize each row of W_E separately according to our choice of $\lambda_1 \in \mathbb{R}^k$

Therefore we illustrate that we can indeed train the autoencoders associated with the different versions of PCA such that the expected properties we have mentioned in the preceding sections do indeed hold in a practical example.

For our illustration we will make use of the *Iris Flower Dataset* [29, 30, 28]. This dataset consists of labeled data⁵ of 50 observations for three different Iris flower species each. Four properties were measured for each observation. Therefore we can define the observation matrix associated with this dataset as $X \in \mathbb{R}^{150 \times 4}$.

Because standard PCA requires pre-processing, we choose to use a scaled and centered version of X as input for both standard PCA and all our versions of autoencoders.

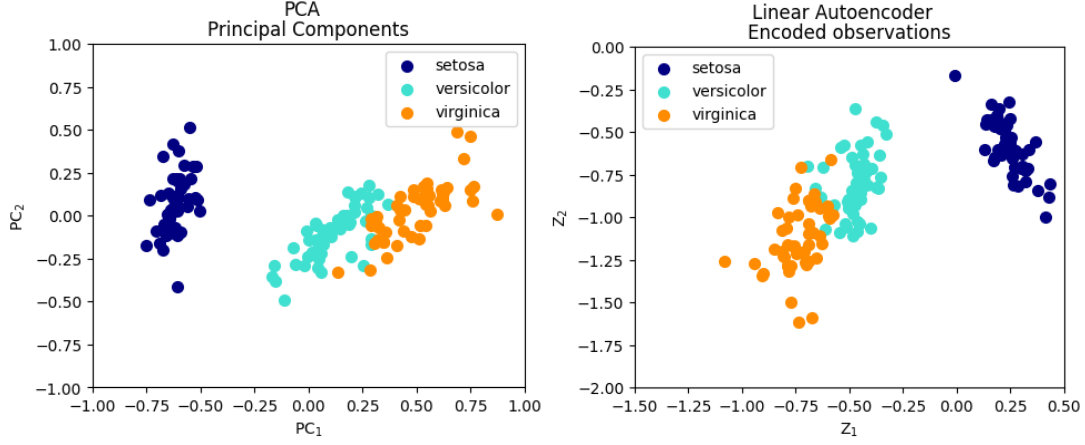
We first scale each entry of the observations in X separately using *MinMax scaling* (Appendix A) such that each observation $x \in [0, 1]^4$. Then, we center X column-wise as in (3.4) to end up with X_0 the scaled and centered version of X . Our pre-processed X_0 is now fit to be applied to standard PCA.

We make use of the SVD of $X_0 = U_{x_0} D_{x_0} V_{x_0}^T$ to find the loadings V_{x_0} and principal components $Z_{x_0} = U_{x_0} D_{x_0}$ for standard PCA.

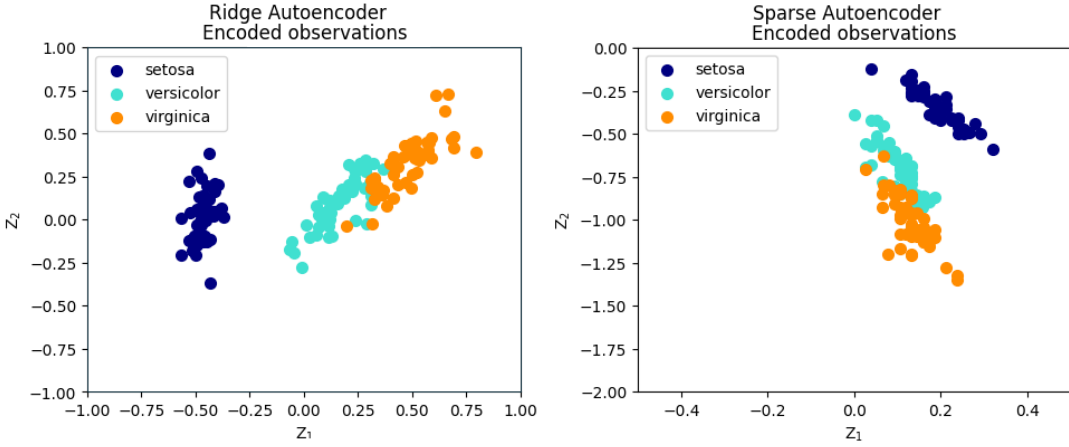
To visualize how PCA represents a linear combination of an observation $x \in [0, 1]^4$ as a representation in terms of the first two principal components $z \in [0, 1]^2$, we plot the first two principal components $Z_{x_0,2}$ of all observations X_0 against each other in Figure 3.1a along with a visualization of V_{x_0} as a heatmap in Figure 3.2a to illustrate the orthogonality of the loading vectors.

We see from Figure 3.1a, as expected, PCA finds a representation in $\mathbb{R}^2$ of all observations X such that we retain enough information to still distinguish between observations of different species of Iris flowers. Also, the loadings are indeed orthogonal as seen in 3.2a.

⁵We make use of a labeled dataset to more clearly distinguish between observations in the figures we produce. We do not otherwise make use of the labels to train our autoencoder.



(a) Standard PCA Principal Components. (b) Linear Autoencoder encoded observations.



(c) Ridge Autoencoder encoded observations. (d) Sparse Autoencoder encoded observations.

Figure 3.1: We apply respectively, standard PCA, the linear autoencoder (3.5), the ridge autoencoder (3.12) and the sparse autoencoder (3.13) to the Iris Flower Dataset. All autoencoders have hidden layer width 2. For standard PCA, the first two principal components are plotted against each other. For the three autoencoders, we plot the encoded observations for each autoencoder.

Next, we train the linear and ridge autoencoders as defined in (3.5) and (3.8) on X_0 ⁶. We choose for both autoencoders, the architecture as $(L, \mathbf{p}) = (1, \{4, 2\}, 4)$ such that the hidden layers have width 2. Furthermore, we make use of the Adam optimizer defined in (2.26) with the choice of hyperparameters: iterations $t = 10\,000$, the learning rate $\eta = 0.001$, decay rates $\beta_1 = 0.9$ and $\beta_2 = 0.999$ and bias $\epsilon = 10^{-8}$. For the λ associated with the ℓ_2 penalty in the ridge autoencoder we choose $\lambda = 0.1$.

⁶We have shown that training the autoencoders in (3.5) and (3.8) on X_0 is equivalent to training a scaled, but not centered X on equivalent autoencoders with trainable encoder and decoder biases.

Equivalent plots to Figure 3.1a can be made, where we plot the rows of the encoded observations $Z = E(X_0) \in \mathbb{R}^{k \times n}$ to visualize the autoencoder equivalents of the first two principal components seen in Figure 3.1a.

We see in Figure 3.1b, that the encoded observations of the linear autoencoder resemble that of the principal components in Figure 3.1a, only they have been rotated via some unknown rotation T .

Alternatively, the encoded observations of the ridge autoencoder in Figure 3.1c are able to identically replicate the principal components in Figure 3.1a without an additional rotation.

We also see from Figures 3.2b and 3.2c, that in both cases the encoder weights are indeed orthogonal, a successful implicit consequence in the case of the linear autoencoder in Figure 3.1b and in the case of the ridge autoencoder, a successful implementation of the orthogonal constraint using skew symmetric matrices described above in Section 3.1.

As mentioned, the added value of the ridge autoencoder lies in the fact that its encoder weight W_{E_r} is nested, a property that the encoder weight of the linear autoencoder W_{E_l} does not have. To illustrate this difference between the two autoencoders, we retrain two new autoencoders, one linear and one ridge, with the same hyperparameters, only now with a hidden layer of width 3.

As the encoded observations of both the linear and ridge autoencoders are uncorrelated, to verify if the encoder weight matrices are nested, it is sufficient to check if the rows of W_E are ordered in terms of variance, i.e.

$$\frac{\|W_{E(1,\cdot)}X_0\|_2^2}{\|W_{E(1,\cdot)}\|_2^2} \geq \frac{\|W_{E(2,\cdot)}X_0\|_2^2}{\|W_{E(2,\cdot)}\|_2^2} \geq \frac{\|W_{E(3,\cdot)}X_0\|_2^2}{\|W_{E(3,\cdot)}\|_2^2}.$$

If this is not the case, we can conclude that W_E is not nested, as the truncated rows of W_E do not maximize $\frac{\|w_i X_0\|_2^2}{\|w_i\|_2^2}$ in descending order when $w_i = W_{E(i,\cdot)}$ for $i < k$.

In our example of the two autoencoders with hidden layer width 3, we see that, for W_E^{lin} , the encoder weight corresponding to the trained linear autoencoder, it holds that

$$\frac{\|W_{E_l(2,\cdot)}X_0\|_2^2}{\|W_{E_l(2,\cdot)}\|_2^2} = 15.5 \leq 20.1 = \frac{\|W_{E_l(3,\cdot)}X_0\|_2^2}{\|W_{E_l(3,\cdot)}\|_2^2}.$$

Therefore the rows of W_{E_l} are not ordered in terms of variance and we can conclude that the trained linear autoencoder does not possess a nested encoder weight.

On the other hand, for W_{E_r} , the encoder weight corresponding the trained ridge autoencoder, it does hold that

$$\frac{\|W_{E_r(1,\cdot)}X_0\|_2^2}{\|W_{E_r(1,\cdot)}\|_2^2} = 90.5 \geq \frac{\|W_{E_r(2,\cdot)}X_0\|_2^2}{\|W_{E_r(2,\cdot)}\|_2^2} = 52.2 \geq \frac{\|W_{E_r(3,\cdot)}X_0\|_2^2}{\|W_{E_r(3,\cdot)}\|_2^2} = 18.5.$$

Implying that W_{E_r} does indeed maximize $\frac{\|w_i X_0\|_2^2}{\|w_i\|_2^2}$ in descending order when $w_i = W_{E_r(i,\cdot)}$ for $i < k$. It is then easy to conclude that the ridge autoencoder possesses a nested encoder weight in W_{E_r} .

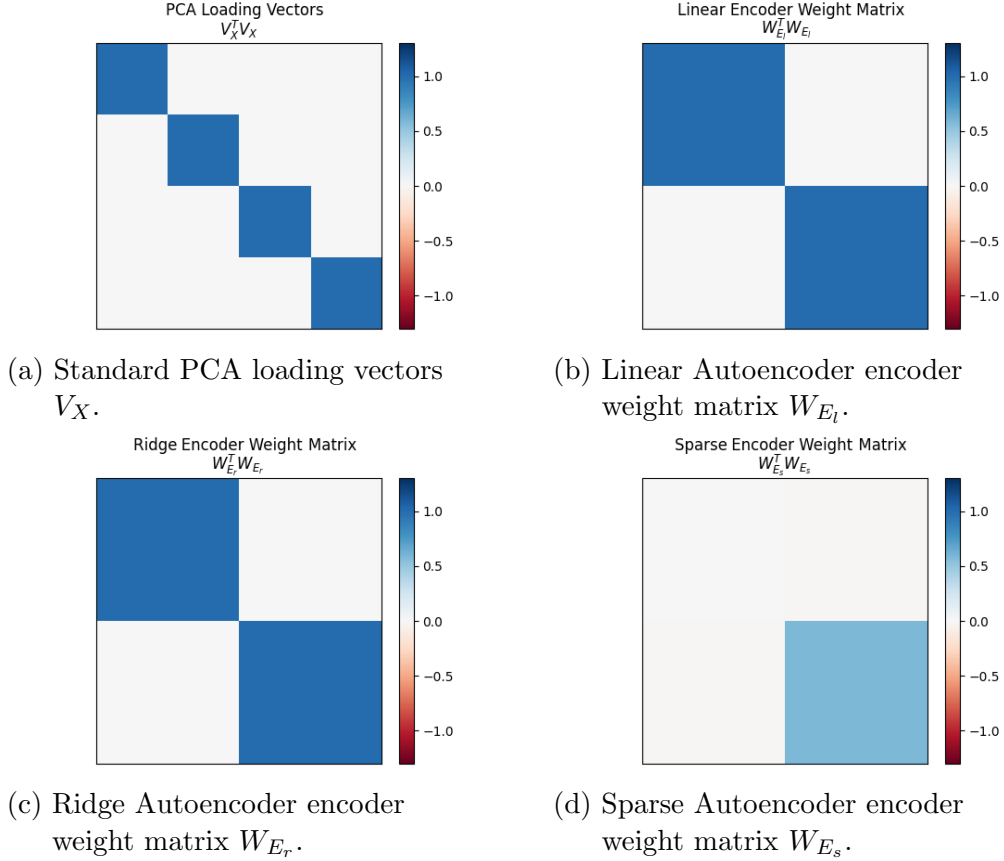


Figure 3.2: A visualization of the matrices $V_x^T V_x$, $W_{E_l}^T W_{E_l}$, $W_{E_r}^T W_{E_r}$, $W_{E_s}^T W_{E_s}$. Here V_x is the matrix of loading vectors computed by standard PCA and W_{E_l} , W_{E_r} , W_{E_s} are the encoder weights for respectively the trained linear autoencoder (3.5), ridge autoencoder (3.12) and sparse autoencoder (3.13). Each entry of a matrix is visualized as a colored block with its color determined by the value of the corresponding entry. An orthogonal matrix is visualized as a diagonal of royal blue block and gray squares not in the diagonal. The sparse autoencoder only has one light-blue block, indicating that $W_{E_s}^T W_{E_s} \neq \mathbf{I}_2$.

A final illustration is in the form of the sparse autoencoder. We again choose the same previous hyperparameters with the addition of $\lambda_1 = [0.3, 0.3, 0.3, 0.3]$. The resulting visualization in Figure 3.1d shows that we are still able to somewhat distinguish the different species of Iris flowers, however the shape of the encoded observations does not resemble that of the principal components from standard PCA. We also note that although the i 'th row of the trained encoder weight $W_{E_s(i,\cdot)} = [0, 0.3, 0, 0]$ is sparse, the resulting encoded observations are not. As a final remark, we see in Figure 3.2d, that the addition of imposed sparsity sacrifices the orthogonality of $W_{E_s(i,\cdot)}$.

3.2 Superposition

The formulations of the autoencoders in (3.5), (3.12) and (3.13) provide us with a way to simulate the existing methods of PCA to a more general framework. However, a great deal of effort is put in to constrain the loadings to be orthogonal, while the orthogonality of the loadings were a consequence of PCA and not a pre-imposed property.

Furthermore, in the context of neural networks and autoencoders, requiring all relations to be linear restricts the power of neural networks, where specifically non-linear relations are easily found with little additional complexity cost. By relaxing these two constraints we return to the more general formulation of an autoencoder. While we lose the convenient interpretations when we are in a linear PCA setting, we are now open to the possibility of learning much more complex and possibly meaningful relationships between components of our data. In the section below, we will see that in the case of sparse data, the addition of only a single non-linear activation function to the linear autoencoder model learns much more complex relationships between components.

The models used and methods of evaluation in the following section are taken from [31]. We do however expand upon the work done in [31] by substantiating empirical conclusions made, with analytic solutions.

The generality of using autoencoders as some sort of "non-linear PCA" opens many doors for exploration of possible new properties to arise when learning new models. One such property is that of so-called *superposition*. To showcase this phenomenon, a toy-model is sketched, where many assumptions are made on the structure of the data. Most notably, on the sparsity of the data, as well as the importance that each component of the data has to the reconstruction thereof.

3.2.1 Linear Toy-Model

To sketch this toy-model, we consider an experiment consisting of n i.i.d. observations where each entry j of an observation $x_i \in [0, 1]^p$, for $i \in \{1, \dots, n\}$, is expressed in terms of two random variables as

$$x_{i,j} = \delta_{i,j} Y_{i,j}, \quad \text{for } \delta_{i,j} \sim \text{Bernoulli}(s) \text{ and } Y_{i,j} \sim U[0, 1].$$

Each entry $x_{i,j}$ is therefore uniformly distributed between 0 and 1 with probability s and zero with probability $1 - s$.

We then want to train a specific autoencoder on the observation matrix $X = [x_1, \dots, x_n]^T \in \mathbb{R}^{n \times p}$ with network architecture $(L, \mathbf{p}) = (1, \{p, k\}, \{p\})$ for some $k < p$. We define the network function of our autoencoder as

$$f : \mathbb{R}^p \rightarrow \mathbb{R}^p, \quad x \mapsto f(x) = WW^T x^T + b.$$

Hence the activation functions are linear, the encoder

$$E : \mathbb{R}^p \rightarrow \mathbb{R}^k, \quad x \mapsto W^T x^T$$

has no bias to train and the decoder

$$D : \mathbb{R}^k \rightarrow \mathbb{R}^p, \quad z \mapsto Wz + b$$

has weight matrix equal to the transpose of the encoder weight, i.e. $W_E^T = W_D =: W$.

Using these definitions, we can define the reconstruction loss as

$$\begin{aligned} \arg \min_{W, b} \mathcal{L}(X, f(X)) &= \arg \min_{W, b} \left\| \Omega_p (X^T - f(X)) \right\|_F^2 \\ &= \arg \min_{W, b} \left\| \Omega_p (X^T - WW^T X^T + b \mathbf{1}_n^T) \right\|_F^2, \end{aligned} \quad (3.14)$$

where $\mathbf{1}_n$ is a column vector of ones of length n and Ω_p is a diagonal $p \times p$ matrix with descending diagonal entries, i.e. $\omega_{1,1} > \omega_{2,2} > \dots > \omega_{p,p}$. This matrix Ω_p serves to weigh the importance that each entry of x has in the experiment, i.e. during training we penalize our model more heavily if a specific entry of x with large importance is not estimated correctly. Our assumption, by choosing Ω_p as above, is therefore that the entries of x are already ordered in descending order of importance and that successive entries cannot be equally important in the experiment.

We immediately see from the derivations made in Section 3.1.1 regarding the linear autoencoder, that the resulting $\widehat{W}$ that minimizes (3.14) here is also orthogonal and consequently, each column $\widehat{W}_{(:,i)}$ of $\widehat{W}$ has the property

$$\|\widehat{W}_{(:,i)}\|_2 := \|\widehat{W}_i\|_2 \in \{0, 1\}.$$

Also, it follows from Section 3.1.1 that the optimal bias in (3.14) can be explicitly computed as

$$\hat{b} = 1/n(X^T - \widehat{W}\widehat{W}^T X^T)\mathbf{1}_n.$$

If we subsequently choose each entry $\omega_{i,i}$ in the diagonal of Ω_p as

$$\omega_{i,i} = \alpha^i,$$

for some $\alpha \in (0, 1)$ such that the importance of the first k entries of each observation x are so important that the impact of the remaining $p - k$ entries on the loss is negligible. We can influence the structure of the resulting optimal $\widehat{W}$ such that

$$\widehat{W}\widehat{W}^T = \begin{bmatrix} \mathbf{I}_k & \mathbf{0}_{k,p-k} \\ \mathbf{0}_{k,p-k} & \mathbf{0}_{p-k,p-k} \end{bmatrix}.$$

This essentially enforces the reproduction $X^\sharp = f(X)^T$ of X to learn

$$\begin{aligned} f(X) &= \widehat{W}\widehat{W}^T X^T + \hat{b} \\ &= \begin{bmatrix} X_{(1:k,\cdot)}^T \\ \mathbf{0}_{p-k,n} \end{bmatrix} + \frac{1}{n} \left(X^T - \begin{bmatrix} X_{(1:k,\cdot)}^T \\ \mathbf{0}_{p-k,n} \end{bmatrix} \right) \mathbf{1}_n \\ &= \begin{bmatrix} X_{(1:k,\cdot)}^T \\ \mathbf{0}_{p-k,n} \end{bmatrix} + \frac{1}{n} \begin{bmatrix} \mathbf{0}_{k,n} \\ X_{(k+1:p,\cdot)}^T \end{bmatrix} \mathbf{1}_n \\ &= \begin{bmatrix} X_{(1:k,\cdot)}^T \\ \mathbf{0}_{p-k,n} \end{bmatrix} + \begin{bmatrix} \mathbf{0}_{1,n} \\ \mu_{X_{k+1}} \\ \vdots \\ \mu_{X_p} \end{bmatrix} \\ &= \begin{bmatrix} X_{(1:k,\cdot)}^T \\ \mu_{X_{k+1}} \\ \vdots \\ \mu_{X_p} \end{bmatrix} \end{aligned}$$

where $\mu_{X_j} = 1/n \sum_{i=1}^n x_{i,j}$, the empirical mean of the j 'th entries of all the observations in X . Then for every $x \in X$ the autoencoder identically reconstructs the first k entries of x and reproduces the empirical mean of each respective component X for the remaining $p - k$ entries of x .

Constructing this artificial model as such has allowed us to recreate some form of PCA where it still remains that $\widehat{W} = V_{x,k}T$ for some orthogonal matrix T but we have artificially enforced an additional ordering in the encoder weight W to embed the ordered importance of each entry in X in the encoded space. If we wanted to assess how the autoencoder encodes and subsequently decodes X we can study $\widehat{W}$ to determine how the learned linear combinations of X contribute to the encoding of X .

To better illustrate the evaluation of $\widehat{W}$, we will use a running example, where each observation x is generated entry-wise, where each entry $x_i \in x$ is i.i.d according to

$$x_i = \delta_{1,i} Y_i, \quad \text{for } \delta_{1,i} \sim \text{Bernoulli}(1) \text{ and } Y_i \sim U[0, 1],$$

such that the observations in X are dense. Further, the encoded space in the hidden layer is $\mathcal{Z}_2 = \mathbb{R}^2$ and for Ω_5 , each diagonal entry $\omega_{i,i} = 0.5^i$. We then train the model with architecture $(L, \mathbf{p}) = (1, \{5, 2\}, \{5\})$ and optimization task defined in (3.14).

We again make use of the Adam optimizer with hyperparameters $t = 10\,000$, $\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.99$, $\epsilon = 10^{-8}$ and $batchsize = 1024$.

After training, we find that indeed

$$\widehat{W}\widehat{W}^T = \begin{bmatrix} \mathbf{I}_2 & \mathbf{0}_{2,3} \\ \mathbf{0}_{3,2} & \mathbf{0}_{3,3} \end{bmatrix}, \quad \hat{b} = \begin{bmatrix} 0 \\ 0 \\ \mu_{X_3} \\ \mu_{X_4} \\ \mu_{X_5} \end{bmatrix}, \quad \begin{bmatrix} \|W_1\|_2 \\ \|W_2\|_2 \\ \|W_3\|_2 \\ \|W_4\|_2 \\ \|W_5\|_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}.$$

We can interpret $W_{(i,j)}$, the i 'th entry of the j 'th column of the encoding matrix, as how we encode x_j , the j 'th entry of an observation x into z_i the i 'th entry of $z = E(x)$, or

$$z_i = \sum_{j=1}^p W_{(i,j)} x_j.$$

Therefore $W_{(\cdot,j)}$ determines how x_j is encoded into each of the k entries of $z \in \mathbb{R}^k$. Similarly,

$$\|W_j\|_2 := \|W_{(\cdot,j)}\|_2$$

then indicates how much of x_j is encoded into all k dimensions of $\mathcal{Z} = \mathbb{R}^k$, the encoded space. If $\|W_j\|_2 = 1$ then x_j is fully encoded into $\mathcal{Z}$.

In our current model we have that $\widehat{W}$ is orthogonal and the norms of each column of $\widehat{W}$ is as follows:

$$\begin{aligned} \|\widehat{W}_1\| &= \|\widehat{W}_2\| = 1 \\ \|\widehat{W}_3\| &= \|\widehat{W}_4\| = \|\widehat{W}_5\| = 0. \end{aligned}$$

Therefore, both x_1 and x_2 are fully encoded into $\mathcal{Z}_2$, such that x_1 and x_2 are orthogonal in $\mathcal{Z}_2$. The remaining entries x_3, x_4, x_5 are subsequently not encoded into $\mathcal{Z}_2$.

Another useful insight $\widehat{W}$ can give us, is how the correlation matrix $\widehat{W}\widehat{W}^T$ influences the reconstruction

$$x^\sharp = f(x) = WW^Tx$$

of x . We can interpret each entry of $(WW^T)_{(i,j)}$ as how the entry x_j is used to reproduce the entry $x_i^\sharp$ of the reconstruction, or

$$x_i^\sharp = \sum_{j=1}^p (WW^T)_{(i,j)} x_j + b_j.$$

In our example, we see that $\widehat{W}\widehat{W}^T$ is an $\mathbf{I}_2$ identity matrix padded with zero block matrices. This confirms that indeed the reconstruction $x^\sharp$ only identically reconstructs the first 2 most important entries of x as $x_i^\sharp = (WW^T)_{(i,i)} x_i = x_i$ for $i \in \{1, 2\}$ and uses the empirical mean of the remaining 3 less important entries of x to reconstruct the remaining entries of $x^\sharp$ as $x_{i'}^\sharp = b_{i'} = \mu_{X_{i'}}$ for $i' \in \{3, 4, 5\}$.

We visualize all relevant values $\widehat{W}$, $\widehat{W}\widehat{W}^T$, $\hat{b}$ and $\|\widehat{W}_1\|_2, \dots, \|\widehat{W}_5\|_2$ in Figure 3.3 as heatmaps for $\widehat{W}$, $\widehat{W}\widehat{W}^T$ and $\hat{b}$ and a horizontal barplot for $\|\widehat{W}_1\|_2, \dots, \|\widehat{W}_5\|_2$. We also plot the two directions in which the first two columns of $\widehat{W}$ embed the entries of x .

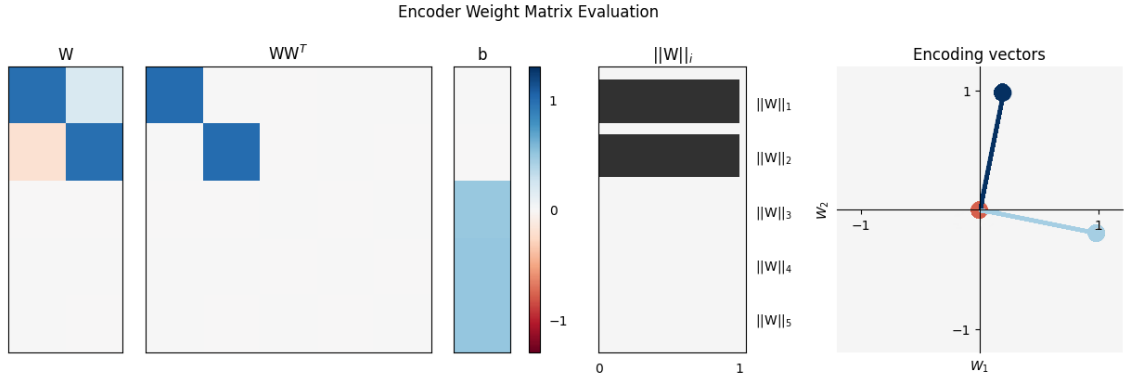


Figure 3.3: Visualization, for trained values of W, b , of W, WW^T, b as heatmaps, the norms of each W_i , the columns of W as a barplot and each W_i plotted against each other.

We see that, as expected, the two royal blue blocks in the second figure indicate that WW^T is an identity matrix padded with zeroes, the light blue blocks in the third figure indicate the bias is either 0 or the empirical mean, each column of W has either norm 0 or 1 and W produces orthogonal directions for encoding.

The next step we can make, is to see if we train our autoencoder for different values of s , data with differing levels of sparsity, whether the trained autoencoder will behave differently.

To visualize the behavior of the linear autoencoder, we plot in Figure 3.4 the same values as in Figure 3.3 for linear autoencoders trained on data with respectively $s = [1, 0.5, 0.25, 0.05]$, such that the data ranges from dense to very sparse.

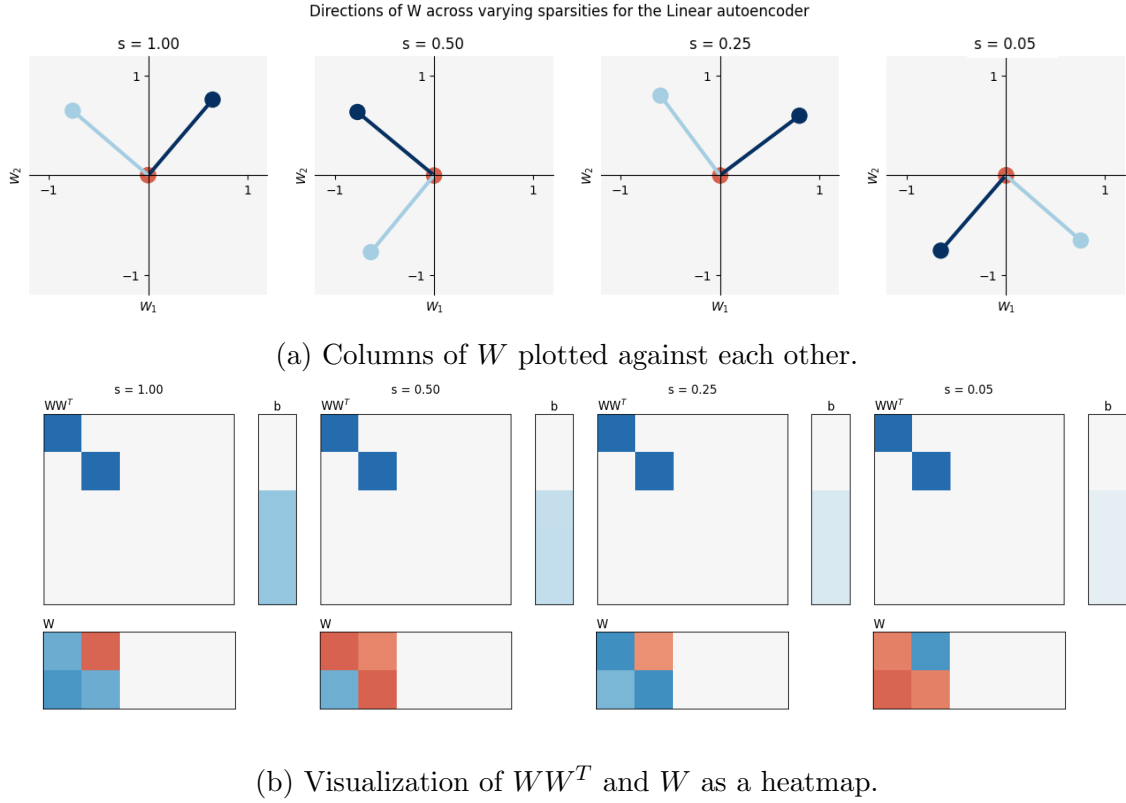


Figure 3.4: Visualization, for trained values of W, b corresponding to $s = [1, 0.5, 0.25, 0.05]$. Again we plot each W_i plotted against each other and W, WW^T, b as heatmaps.

We see that, for different values s the linear autoencoder learns different directions in W , however the resulting WW^T remains an identity matrix padded with zeroes. This indicates that the reproduction of the linear autoencoder remains identical across different levels of sparsity.

This plot shows that although we might find different $\widehat{W}$'s for different sparsities, the columns of W remain orthogonal and the resulting reproduction remains identical. What we will see however, is that this changes with the addition of a single ReLU activation.

3.2.2 ReLU Toy-Model

In an attempt to change the behavior of the autoencoder in the presence of sparsity, we now adjust our autoencoder to include a non-linear ReLU activation function in the decoder. The optimization task is adjusted to

$$\begin{aligned}\mathcal{L}(X, f(X)) &= \arg \min_{W, b} \left\| \Omega_p(X^T - \sigma_b W W^T X^T) \right\|_F^2, \\ &= \arg \min_{W, b} \left\| \Omega_p(X^T - \text{ReLU}(W W^T X^T + b)) \right\|_F^2\end{aligned}\tag{3.15}$$

where $\sigma_b(x_i) = \text{ReLU}(x_i + b) = \max\{0, x_i + b\}$ is the shifted ReLU activation function. We do not analytically compute the true optima for W and b in (3.15), but instead extend our example from the previous section to the ReLU autoencoder.

In Figure 3.6 we can see that in the case of dense data, the addition of the ReLU activation does not change the structure of $\widehat{W}$, so that $\widehat{W}\widehat{W}^T$ is still $\mathbf{I}_2$ padded with zeros and the optimal $\hat{b}$ equals that of (3.14), for the linear autoencoder. Therefore the ReLU autoencoder and linear autoencoder identically reconstruct any dense observation x .

Now, for $s < 1$, we see in Figure 3.6 that the ReLU model behaves differently. It is not the case anymore that the autoencoder only retains information of the first 2 components. The autoencoder now still learns to mostly reproduce the identity in the first 2 components. However, the ReLU autoencoder also starts to add information from the remaining 5 components in the form of linear combinations in the reproduction, such that

$$x_j^\# = \text{ReLU} \left(\sum_{i=1}^5 (\widehat{W}\widehat{W})_{(i,j)}^T x_i + \hat{b} \right),$$

for each entry x_j of a sparse observation x . This is in contrast to

$$x_j^\# = \text{ReLU} \left((\widehat{W}\widehat{W})_{(j,j)}^T x_j + \hat{b} \right)$$

when x is dense.

We also see in Figure 3.6 that this behavior increases when we decrease s , such that the sparsity of X increases. Looking specifically at each $\|W_i\|$ to gauge how much of each entry x_i of x is encoded, we also now see that the first 2 components are still mostly encoded, but the remaining 3 components make start to additionally contribute to the encoding. In general, the sum of the norms $\sum_{i=1}^5 \|\widehat{W}_i\|_2$ increases with the level of sparsity in the data. This implies that more information of x is encoded in the ReLU autoencoder as the sparsity increases. Also the number of components used in the encoding increases along with the sparsity.

To ensure that the deviating behavior of the ReLU autoencoder compared to the linear autoencoder is actually desired, we plot the loss functions of each autoencoder as a function of s in Figure 3.5, to see that indeed, with sparse data, our ReLU autoencoder produces a smaller reproduction loss and therefore is able to more accurately reproduce sparse observations x compared to the linear autoencoder.

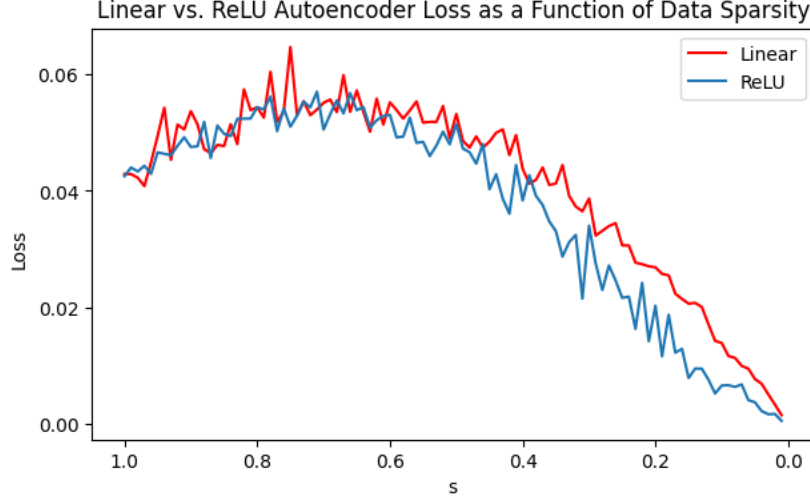


Figure 3.5: Plot of the loss functions of the linear autoencoder (red) defined in (3.14) and the ReLU autoencoder (blue) defined in (3.15). We see that the ReLU autoencoder has, in general, for smaller values of s (more sparsity), a smaller reconstruction loss.

Superposition

The properties that have arisen with the addition of the ReLU activation indicate that when our data is sparse, the non-linear relationship we have imposed in the reconstruction, results in learning new relationship from the data that the linear model is not able to learn. This phenomenon is called *superposition*. A more formal definition follows

Definition 3.1 (Superposition). *Superposition* refers to how much the direction of a column $\|W\|_i$ in the weight matrix of an autoencoder is influenced by the other columns $\|W\|_j$ for $i \neq j$. The superposition of $\|W\|_i$ can be computed by taking the norm of all other columns W_j for $i \neq j$ projected onto W_i :

$$\sum_{i \neq j} [(W_i)^T W_j]^2.$$

This represents how much of the other columns W_j are represented in W_i .

The definition of superposition now allows us to better interpret the results of Figure 3.6.

The linear and sparsity-free ReLU models show us that each column W_i is either 0 or orthogonal to all other columns. However with the increase of sparsity, the ReLU model replaces a column W_j consisting of zeros with a non-zero W_i in a specific direction.

In Figure 3.6a we see the easiest additional direction to spot is W_i such that $W_i = -aW_j$ for some $a > 0$ and $i \neq j$. The addition of a vector in this direction can be explained by the use of the ReLU activation.

We can encode extra information in the form of negative correlation, i.e. add a W_i such that $W_i^T W_j < 0$. With the assumption that $x \in [0, 1]$, we do not incur an extra penalty in the loss (3.15), as

$$\sigma_b(W_i^T W_j x) \leq \sigma_b(0), \quad \text{for } \sigma_b(x) = \text{ReLU}(x) + b.$$

To ensure that the addition of W_i does not incur an additional penalty in other components, we must ensure that $W_i^T W_{j'} = 0$ for all $j' \neq i \neq j$. This results in W_i being orthogonal to all other $j' \neq j \neq i$ columns of W . Therefore the resulting additional $W_i = -aW_j$ will be antipodal to some other non-zero W_j . We see in Figure 3.6a, that these antipodal directions are added first when s is still relatively large (little sparsity).

The gain in information with the addition of further W_i 's seems to override the increase of the reconstruction penalty when adding positively correlated components.

We do however notice that after all antipodal directions have been added in the columns of W , additional directions start to influence the directions of existing columns such that a different form of superposition takes place.

If the resulting directions in W are influenced by each other, one would assume that there is an optimal arrangement for all directions. To find this arrangement we notice that when $W_i^T W_j > 0$ the resulting loss increases. Therefore it seems logical to find directions for each W_i such that each pair $W_i^T W_j < \epsilon$ for $i \neq j$ and some small $\epsilon > 0$. This implies that it would be ideal for the components of W to be as mutually orthogonal as possible.

We see in Figure 3.6a that when there are more non-zero columns of W than there are possible antipodal directions, the directions of the columns are arranged such that each column is almost orthogonal to the adjacent columns in the figure. We also see that this additional phenomenon arises in W for specifically very small values of s , i.e. when the data is very sparse.

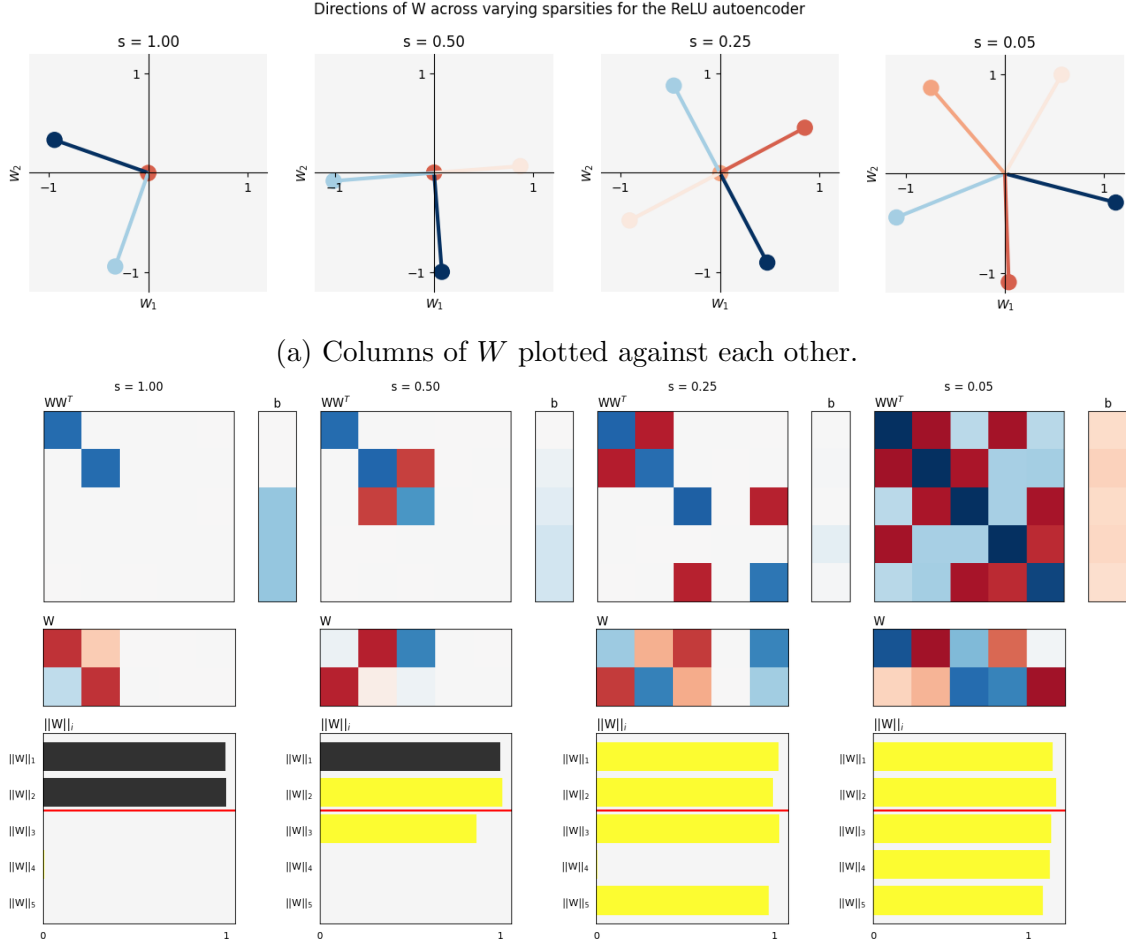


Figure 3.6: We see that, as the level of sparsity increases, superposition takes place in W . Initially with the addition of antipodal directions and then an adjusted arrangement such that all columns are almost orthogonal to their neighbors. We also see in the barplot of each $\|W_i\|$ for $s = 0.5$, that the addition of W_3 in an antipodal direction introduces superposition between W_2 and W_3 while W_1 remains unaffected. With the addition of W_5 for $s = 0.25$, the barplot, in combination with the heatmap for WW^T , indicates that now W_1 and W_2 are in superposition as well as W_3 and W_5 . When $s = 0.05$ the heatmap of WW^T indicates that all columns are in superposition with each other.

We see, in conclusion, that for sparse data, the addition of a non-linear relationship changes the encoding and subsequent decoding of data such that the autoencoder more accurately reproduces observations.

Also the encoding learns to combine entries of x that are otherwise independent, in a non trivial way to increase the amount of information that is encoded into the encoded space $\mathcal{Z}$. It is however important to notice that while this approach improves the numerical performance, it loses the interpretive power of the linear autoencoder.

In practical applications however, the loss of interpretability seems to outweigh the increase in numerical performance. This loss of interpretability is enough of a problem that follow-up work of [31] attempts to "solve" superposition as if it is an unwanted property and deems it worthwhile to sacrifice performance in exchange for interpretability of the encoded observations [31].

3.2.3 Compressive Sensing

Another consequence of the autoencoder model in combination with sparse data we have described in this section is that we can make a link to compressive sensing. Such that we can use results of compressive sensing to infer properties on the width of the hidden layer that ensures a small reconstruction error. To accomplish this, we must first reformulate a few concepts from Section 2.1.1 to better resemble the framework of autoencoders.

The alternative formulation of the compressive sensing problem in (2.5), is stated as determining a suitable sensing matrix $A \in \mathbb{R}^{k \times n}$ and sparsity promoting map $\Delta : \mathbb{R}^k \rightarrow \mathbb{R}^p$ such that the reconstruction error for any vector $z \in \mathbb{R}^p$

$$\mathcal{L}(y) = \|z - \Delta(y)\|_p = \|z - \Delta(Az)\|_q, \quad (3.16)$$

is minimized for some $q \geq 1$. We easily see that, for the map

$$\Delta_{1,A}(y) := \arg \min \{\|z\|_1 : Az = y\},$$

the basis pursuit algorithm in (2.7) can be expressed as minimizing the reconstruction error

$$\|z - \Delta_{1,A}(Az)\|_q.$$

To state the above minimization problem as the autoencoder from the toy-model proposed in (3.15), we set the encoder

$$E(x) = W^T x = Ax,$$

such that the weight matrix $W = A^T$. We additionally set the weighted decoder as

$$\Omega_p D(y) = \Omega_p \text{ReLU}(y + b) = \Omega_p \sigma_b(Wy) = \Delta(y)$$

and the loss function to the ℓ_2 loss.

Then, one would assume that the compressive sensing reconstruction error in (3.16) can be reformulated as the loss function of the optimization task of the ReLU autoencoder evaluated at a single s -sparse observation x as follows

$$\mathcal{L}(\theta_e, \theta_d) = \|x - \Omega_p \sigma_b(WW^T x)\|_2^2 = \|x - \Delta(Ax)\|_2^2 = \mathcal{L}(y). \quad (3.17)$$

The key difference between the two approaches is that in compressive sensing we know the measurements $y = Az$, are free to choose a suitable matrix A and are tasked to learn a reproduction $z^\#$ of a sparse z .

In the case of autoencoders, we do not beforehand know a good choice for W , the autoencoder analog of A , as we have to first train the autoencoder to learn the optimal W that reconstructs x . Also, we are given x , the analog of z , to train the autoencoder on. Therefore, there is nothing to learn in the context of compressive sensing.

Hence, we cannot use compressive sensing techniques on problems stated as autoencoders. We are instead, restricted to evaluating, after training has been completed, if properties of the optimal $\widehat{W}$ satisfy results that can claim low reproduction errors or if the ability of a specific autoencoder to reproduce a sparse observation x is comparable to that of a compressive sensing algorithm.

To evaluate the performance of our autoencoder, we notice a problem in the fact that the resulting reconstruction $x^\# = \sigma_b(W^T W x)$ does not necessarily reproduce a s -sparse vector as is required in compressive sensing. To ensure that we can indeed state (3.17) as a compressed sensing problem we introduce concept of *optimal recovery*:

Definition 3.2 (Instance Optimality [10]). Given $q \geq 1$, a pair (A, Δ) , consisting of a measurement matrix $A \in \mathbb{R}^{n \times p}$ and reconstruction map $\Delta : \mathbb{R}^k \rightarrow \mathbb{R}^p$ is called ℓ_q -instance optimal of order s with constant $C > 0$ if,

$$\|z - \Delta(Az)\|_q \leq C \phi_s(z)_q,$$

for any $z \in \mathbb{R}^p$, where

$$\phi_s(z)_q = \inf\{\|z - z_s\|_q, \quad z_s \in \mathbb{R}^p \text{ is } s\text{-sparse}\}.$$

Therefore the pair (A, Δ) *optimally recovers* a vector $z \in \mathbb{R}^p$ in ℓ_p , if the ℓ_p reconstruction error is smaller than the best s -sparse reconstruction of z times some constant C .

The pair $(W^T, \Omega_p D(\cdot))$ in our proposed autoencoder algorithm then optimally recovers any observation $x \in \mathbb{R}^p$ in ℓ_2 , if the reconstruction error has upper bound

$$\|x - \Omega_p \sigma_b(WW^T x)\|_2^2 \leq C \phi_s(x)_2.$$

This means that if the autoencoder we train performs well enough in the reconstruction error, we have successfully implemented a compressive sensing algorithm as an autoencoder.

We can go a step further by imposing conditions on W as well as the reconstruction error of the autoencoder to infer a lower bound on the size of the hidden layer width of an autoencoder that ensures a good reconstruction of sparse observations x .

Theorem 3.3 (Theorem 2 of [31]). *For W, b the trained encoder weight and bias of the autoencoder defined in (3.15).*

If W has the (δ, s) restricted isometry property and W, b are such that, for any $x \in \mathbb{R}^p$, it holds that

$$\|x - \Omega_p \sigma_b(WW^T x)\|_2^2 \leq C\theta_s(x)_2.$$

Then we can lower bound the width of the hidden layer as $k \geq C's \ln(n/s)$ for some constant $C' > 0$.

Although these restrictions are substantial, especially the fact that constructing matrices that satisfy the restricted isometry property (Appendix A) remains an open problem in the field of compressed sensing [10]. The fact that we can concretely infer properties on components in the architecture of our autoencoder from results in the field of compressed sensing provides a promising link between the two fields.

It is important to note that, although there is a link between autoencoders and compressed sensing when the observations are sparse, compressed sensing algorithms use much more general techniques, while autoencoders are bound to the structure of neural networks. Therefore compressed sensing techniques are generally more powerful in terms of sparse recovery, while autoencoders in turn are much broader in their application such that sparse recovery is only one of many applications the framework of autoencoders can be applied to.

4 Discussion

The expression of the different PCA minimization problems presented in [16] as autoencoders is largely motivated as a replication of existing techniques. However, applying PCA as an autoencoder has the major benefit of scaling to datasets with more observations with little additional computation cost.

In contrast, increasing the number of observations in the dataset complicates standard PCA in that the SVD of an ever increasing matrix $X \in \mathbb{R}^{n \times p}$ must be computed. The complexity of training autoencoders on the other hand does not scale with sample size n as, during training, minibatches are used such that for each iteration t of the training process, the autoencoder is trained on a subset $X_t \subset X$ with a fixed sample size of $n' < n$ such that $X_t \in \mathbb{R}^{n' \times p}$.

With no extra computation cost, the performance in terms of reproduction error of autoencoders actually increases with sample size due to the fact that a larger sample size results in more unique combinations of minibatches X_t to train on. Therefore the autoencoder is less likely to converge to a local optimum during training. The converse is however true for autoencoders applied to datasets with fewer observations as it is easier to get stuck in a local optimum when training on the same or similar observations for each step t in the training process.

The main benefit of this formulation is that the task of PCA is now part of a larger framework capable of implementing multiple other techniques. A concrete example is the implementation of Principal Component Regression (PCR), where instead of performing linear regression on the (possibly high-dimensional) observation matrix X , you perform linear regression on the principal components Z_x and then transform the learned regression parameter $\beta_k \in \mathbb{R}^k$ in the space spanned by the first k principal components via the loading vectors to the regression parameter $\beta_p \in \mathbb{R}^p$ in the space spanned by the observations in X .

One could similarly implement PCR, for $Y \in \mathbb{R}^n$ the vector of responses, as a neural network by either training a neural network with architecture $(0, \{k, 1\})$ and loss function

$$\arg \min_{W_\beta, b_\beta} \|Y - W_\beta Z + b_\beta\|_F^2,$$

on the encoded observations $Z \in \mathcal{Z}$ of an already trained autoencoder. A response $\hat{y}$ corresponding to the observation x can then be estimated as

$$\hat{y} = \widehat{W}_\beta E(x) + \hat{b}_\beta \mathbf{1}_n^T,$$

where $E(x)$ is x encoded through the trained autoencoder and $\widehat{W}_\beta, \hat{b}_\beta$ are the trained parameters of the regression neural network.

Another more advantageous implementation could be to alter the architecture of the autoencoder as a composite neural network that learns the encoded regression parameters and the encoder weights simultaneously. This can be done by simply adding a regression term in the loss function of the autoencoder such that the optimization task of the composite neural network becomes

$$\arg \min_{\theta_e, \theta_d, W_\beta, b_\beta} \mathcal{L}(\theta_e, \theta_d) + \mathcal{L}(W_\beta) = \arg \min_{\theta_e, \theta_d, W_\beta, b_\beta} \|X^T - D(E(X))\|_F^2 + \|Y^T - W_\beta X^T + b_\beta \mathbf{1}_n^T\|_F^2.$$

This way we have easily implemented PCA and PCR simultaneously as a single neural network.

With autoencoder PCA as a baseline we can then attempt to extend methods that make use of PCA into neural network frameworks such that the modularity of neural networks can be efficiently exploited as can be done with the implementation of PCR.

The more powerful extension of PCA that autoencoders provide is the addition of non-linear feature extraction. As mentioned in the introduction, non-linear feature extraction is not a new concept. However, all variations of PCA that incorporate non-linearity, still require that the principal components are a linear combination of the entries of some non-linear transformation of an observation x . Autoencoders easily and successfully incorporate non-linear relationships between the observations and the encoded observations. Therefore, autoencoders uniquely provide non-linearity to the idea of PCA.

The advantage of these non-linear relationships is present in the formulation of the toy-model in (3.15) where the data is made to be sparse. The non-linear ReLU activation learns more meaningful encodings in the form of superposition. As seen in Figure 3.5, these encodings subsequently produce a better reproduction of x compared to standard PCA (implemented as a linear encoder) as a result.

It seems however that in [31], where this phenomenon of superposition was first investigated, a need¹ for balance between interpretability and performance is desired and superposition is treated as an undesirable property that needs to be overcome at the cost of a minimal increase in reproduction error.

Another hurdle to overcome is choosing appropriate non-linearities for specific situations. We studied an adjusted version of the toy-model in Section 3.2, where instead of data being sampled in $[0, 1]^p$, essentially a hypercube of dimension p , we sampled data, uniformly distributed inside 3-dimensional polygons, initially the 5 platonic solids and later polygons formed by evenly spacing $a \geq 4$ points on the 3-dimensional sphere. Unfortunately, the addition of a ReLU non-linearity was not able to improve on the linear autoencoder replicating PCA.

¹At least in the field of Natural Language Processing

Neither the linear nor the ReLU autoencoders were able to learn meaningful directions in the weight matrix W , other than forming a rotated orthogonal basis to encode the observations onto. Therefore using a ReLU activation here is not as fruitful as in the case of sparse data.

Regarding the connection between compressive sensing and autoencoders. Although autoencoders allow us to make reconstructions of sparse observations, we still rely on being given the observations that would have otherwise been unknown in a compressive sensing setting. This requirements considerably limits the usefulness of autoencoders as compressive sensing algorithms.

Using the theory of compressive sensing however is a useful tool in the analysis of suitable autoencoder architectures that are able to reproduce sparse observations with small reconstruction errors. A limitation of this approach is that many theorems of compressive sensing require properties that hold for all possible s -sparse observations $x \in \mathbb{R}^p$, for $s < p$. However, the nature of autoencoders are that they are constructed and trained for a specific input X . Therefore, we are not able to make any conclusions on the quality of a specific autoencoders for all possible s -sparse observations x . This implies that, verifying the properties needed to apply theorems from the field of compressive sensing, do hold in practical applications, would be challenging.

5 Future Work

Our work largely consisted of considering the most basic constructions of autoencoders. While the addition of non-linear encoders and decoders make autoencoders a powerful tool, it is the generality of the structure of autoencoders that allow autoencoders to be as widely implemented as they are. Therefore the obvious continuation of this work would be to explore the different variations of autoencoder structures available.

Specifically *Variational Autoencoders* (VAE's) are widely implemented [1]. The key difference is that VAE's are instead trained to learn parameters θ_e, θ_d such that the encoder $E_q(x)$ samples from a distribution with density $q(z) := \mathbb{P}(z|x, \theta_e)$ and the decoder $D_p(z)$ then reconstructs x by sampling from a distribution with density $p(x^\#) := \mathbb{P}(x^\#|z, \theta_d)$. The corresponding optimization task then becomes

$$\arg \min_{\theta_e, \theta_d} -\frac{1}{2} \|X - D_p(E_q(X))\|_F^2 - D_{KL}(q \parallel p),$$

where $D_{KL}(\cdot \parallel \cdot)$ is the Kulbeck-Leibler divergence.

Hence VAE's learn parameters of distributions instead of deterministic functions [21]. This formulation automatically lends itself to connections with Bayesian Statistics.

The apparent success of being able to extend PCA to the framework of autoencoders leads us to believe that it might be possible to extend other feature extraction methods to the autoencoder framework.

In [32], a comparison is made between VAE's and factor analysis. It is however shown that VAE's do not encapsulate the generality of the most general factor analysis problem. At the same, standard PCA is considered a specific case of the more general problem of factor analysis. It would then seem possible that PCA could be generalized to an intermediate case between standard PCA and general factor analysis such that the direct equivalence between a factor analysis problem and a suitable autoencoder is retained.

The toy-model of superposition we investigated, was constructed in [31] from a interpretability point of view in the field of Natural Language Processing. Therefore the results found from this toy-model can be widely applied in this field. Especially, a further understanding of the mechanics of how an autoencoder with ReLU activation interacts with superposition allows to further the interpretability of models with similar structures.

While our implementation of autoencoders applied to data distributed inside polygons was not fruitful, it would seem possible to construct an autoencoder that is able to learn in the encoder weight W_E properties such as the directions of the tangents to the faces of the polygons or the locations of vertices, or even a less straightforward combination of directions in W_E might be possible to identify, which can be interpreted in terms of the structure of the generated data. The more general concept of this idea is the combination of Topological Data Analysis and autoencoders which has been explored in [33].

To circumvent properties in compressive sensing that are required to hold for all s -sparse vectors z , *statistical compressive sensing* has been developed. The main difference is that we assume the unknown observations are distributed by some distribution known to us. This results in equivalent properties of compressive sensing to be stated in terms of the expectation. For example we can equivalently state a statistical compressive sensing version of instance optimality defined in Definition 3.2 as:

A map $\Delta: \mathbb{R}^n \rightarrow \mathbb{R}^p$ is called ℓ_q -instance optimal in expectation of order s if holds

$$\mathbb{E}_z \|z - \Delta(Az)\|_2 \leq C \mathbb{E}_z \phi_s(z)_q,$$

for some constant $C > 0$ [34].

It might then be possible to use results from statistical compressive sensing, as they develop, that are more easily verifiable in practice for autoencoders. Some results in this sense have already been shown for VAE's in [35] and therefore it would not entirely be unreasonable to extend similar results to a more general class of autoencoders.

6 Appendix

Appendix A

Linear Algebra

We start with definitions regarding a square matrix $A \in \mathbb{R}^{p \times p}$ [36]:

- **Orthogonal matrix:** A is called *orthogonal* if $A^T A = \mathbf{I}_p$ for the $\mathbf{I}_p$ the $p \times p$ identity matrix. This implies that $A^{-1} = A^T$,
- **Symmetric matrix:** A is called *symmetric* if $A = A^T$.
- **Skew-symmetric matrix:** A is called *skew-symmetric* if $A = -A^T$. This implies that $A = A_{\Delta} - A_{\Delta}^T$ for A_{Δ} a matrix where all entries on and below the diagonal are 0.
- **Trace:** The *trace* of A can be defined as

$$\text{tr}(A) = \sum_{i=1}^p A_{(i,i)}.$$

- **Matrix exponential:** The *exponential* e^A of A can be defined as

$$e^A = \sum_{k=0}^{\infty} \frac{1}{k!} A^k.$$

- **Moore-Penrose inverse:** If A is also symmetric, then the *Moore-Penrose inverse* [11] of A can be defined as

$$A^+ = \sum_{j=1}^p \nu_j^{-1} \mathbf{1}_{\{\nu_j \neq 0\}} v_j v_j^T,$$

where ν_j and v_j are, respectively, the eigenvalues and corresponding eigenvectors of A .

Some additional definitions and theorems follow.

The *singular value decomposition* (SVD) of a matrix $X \in \mathbb{R}^{n \times p}$ can be expressed as

$$X = U_x D_x V_x^T.$$

Here D_x is a $n \times p$ block matrix with the upper left block consisting of a $\text{rank}(X) \times \text{rank}(X)$ diagonal matrix with the singular values of X on its diagonal and the rest of D_x is zero up to the appropriate dimensions. The matrix U_x is an $n \times n$ matrix with its columns consisting of the left singular vectors. The matrix V_x , in turn, $p \times p$ matrix with its columns consisting of the right singular vectors. The columns of both U_x and V_x are orthogonal [36].

The *k-thin SVD* of a matrix $X \in \mathbb{R}^{n \times p}$ can be expressed as

$$X = U_{x,k} D_{x,k} V_{x,k}^T,$$

where $U_{x,k} \in \mathbb{R}^{n \times k}$ and $V_{x,k} \in \mathbb{R}^{p \times k}$ are the matrices consisting of the first k columns of U_x and V_x respectively and $D_{x,k} \in \mathbb{R}^{k \times k}$ the diagonal matrix consisting of the first k singular values of X [4].

With the *k-thin SVD* of a matrix defined. We can state the Eckhart-Young-Mirsky theorem[4]:

Theorem 6.1 (Eckhart-Young-Mirsky). *The solution $\widehat{X}_k$ to the minimization problem*

$$\arg \min_{X_k: \text{rank}(X_k) \leq k} \|X - X_k\|_F,$$

*for a matrix $X \in \mathbb{R}^{n \times p}$, is the *k-thin SVD* of $X = U_{x,k} D_{x,k} V_{x,k}^T$. Specifically, $\|X - \widehat{X}_k\|_F^2 = \sum_{j=k+1}^p \sigma_j^2$ for σ_i^2 the *i*'th singular value of X .*

The *Hadamard product* [36] $X \odot Y$ of two matrices $X, Y \in \mathbb{R}^{n \times p}$ can be expressed element-wise as

$$(X \odot Y)_{(i,j)} = X_{(i,j)} Y_{(i,j)} \quad \text{for } i \in \{1, \dots, n\} \text{ and } j \in \{1, \dots, p\}.$$

A vector $x \in \{x_1, \dots, x_n\}$ is scaled according to the *MinMax Scaler* to find a rescaled version $x' \in [0, 1]$ of x as

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}},$$

where $x_{\min} = \min\{x_1, \dots, x_n\}$ and $x_{\max} = \max\{x_1, \dots, x_n\}$ [28].

A matrix $X \in \mathbb{R}^{n \times p}$ satisfies the (δ, s) *restricted isometry property* if, for $\delta \in (0, 1)$, it holds that

$$(1 - \delta)\|z\|_2^2 \leq \|Xz\|_2^2 \leq (1 + \delta)\|z\|_2^2,$$

for all vectors $z \in \mathbb{R}^p$ such that $\|z\|_0 \leq s$ [10].

Norms

We define the relevant norms to this thesis for a vector $x \in \mathbb{R}^p$ and a matrix $X \in \mathbb{R}^{n \times p}$:

- **ℓ_0 -norm:** $\|x\|_0 := \text{card}(\text{supp}(x))$ [10],
- **ℓ_1 -norm:** $\|x\|_1 := \sum_{i=1}^p |x_i|$ [21],
- **ℓ_2 -norm:** $\|x\|_2^2 := \sum_{i=1}^p x_i^2$ [21],
- **Frobenius-norm:** $\|X\|_F^2 := \sum_{i=1}^n \|X_{(i,j)}\|_2^2 = \sum_{i=1}^n \sum_{j=1}^p (X_{(i,j)})^2$ [21].

We additionally define cardinality and support of a vector x [10]:

The *cardinality*, $\text{card}(S)$ of a set S is the number of entries in S .

The *support* of a vector $x \in \mathbb{R}^p$ is the index set of its nonzero entries, i.e.

$$\text{supp}(x) := \{i \in \{1, \dots, p\} : x_i \neq 0\}.$$

Kernels

A *kernel* [8] is any function $K(x) \geq 0$ that satisfies

$$\int K(x)dx = 1, \quad \int xK(x)dx = 0.$$

We can also define

$$\sigma^2 = \int x^2 K(x)dx > 0.$$

A common choice for $K(x)$ is the *Gaussian Kernel*

$$K(x) = \frac{1}{\sqrt{2\pi}} \exp(x^2/2).$$

With a kernel K defined, we state Theorem 2 of [9] in full.

Let $\mathcal{D} \subseteq \mathbb{R}^p$, X a bounded random variable with uniform continuous density g that has bounded uniform continuous first derivatives on $\mathcal{D}$. Then, for Y the response corresponding to X , we define

$$r(x) = \mathbb{E}(Y|X = x), \quad \sigma^2(x) = \mathbb{E}(Y^2|X = x) - r(x),$$

assume that $r(x)$ is uniformly continuous, that the second derivatives of $r(x)$ are bounded and uniformly continuous on $\mathcal{D}$ and that $\sigma^2(x)$ is bounded. We additionally define

$$\gamma(x) = \sum_{j=1}^p \left[\frac{\partial g(x)}{\partial x^{(j)}} \cdot \frac{\partial r(x)}{\partial x^{(j)}} + \frac{1}{2} g(x) \frac{\partial^2 \mu(x)}{\partial (x^{(j)})^2} \right].$$

Then, for a kernel $K_p(x)$, where $x \in \mathbb{R}^p$, we can state the following theorem

Theorem 6.2 (Theorem 2 of [9]). *Under regularity conditions and for $\hat{r}_{h,n}(x)$ the Nadarya-Watson kernel estimator, if $h \rightarrow 0$ and $nh^p \rightarrow \infty$ as $n \rightarrow \infty$ we have*

$$R(h) = \underbrace{h^4 \int_{\mathcal{D}} \frac{\gamma^2(x)}{g(x)^2} dx}_{bias} + \underbrace{\frac{1}{nh^p} \int_{\mathcal{D}} \frac{\sigma^2(x)}{g(x)} dx \cdot \int K_p^2(u) du}_{variance} + o_p \left(h^4 + \frac{1}{nh^p} \right).$$

Bibliography

- [1] K Berahmand and F Daneshfar and E S Salehi and Y Li, *Autoencoders and their applications in machine learning: a survey*, Artificial Intelligence Review, **57**(28) 2024
- [2] Z Li and H Huang and Z Zhang and G Shi, *Manifold-based multi-deep belief network for feature extraction of hyperspectral image*, Artificial Intelligence Review, **14**(6), 14-84, 2022
- [3] A M Martinez and A C Kak, *PCA versus LDA*, IEEE Transactions on Pattern Analysis and Machine Intelligence, **23**(2), 228-233, 2001
- [4] P Be and R Rosipal and L Trejo and A Cichocki, *Kernel Principal Component Regression with EM Approach to Nonlinear Principal Components Extraction*, 2001.
- [5] B Ghojogh and A Ghodsi and F Kararray and M Crowley, *Locally Linear Embedding and its Variants: Tutorial and Survey*, arXiv:2011.10925 [stat.ML], 2011.
- [6] G Hinton and S Roweis, *Stochastic neighbor embedding*, Neural Information Processing, January 2002.
- [7] L Wasserman, *All of nonparametric statistics*, Springer, 2006.
- [8] P Serra, *Lecture Notes Nonparametric Statistics*, March 2023
- [9] P Hall, *Integrated Square Error Properties of Kernel Estimators of Regression Functions*, The Annals of Statistics **12**(1), 241-260, 1984.
- [10] S Foucart and H Rauhut, *A Mathematical Introduction to Compressive Sensing*, Birkhäuser, 2013.
- [11] W N van Wieringen, *Lecture notes on ridge regression*, June 2023.
- [12] H Smith and N R Draper *Applied regression analysis*, Wiley, 2011
- [13] C M Theobald *Generalizations of Mean Square Error Applied to Ridge Regression*, Journal of the Royal Statistical Society: Series B (Methodological) **36**(1), 103-106, 1974.
- [14] R Tibshirani, *Regression Shrinkage and Selection Via the Lasso*, Journal of the Royal Statistical Society: Series B (Methodological), **58**, 267-288, 1996.

- [15] I T Jolliffe, *Principal component analysis*, Springer, 2011.
- [16] H Zou and T J Hastie and R Tibshirani, *Sparse Principal Component Analysis*, Journal of Computational and Graphical Statistics, **15**, 265-286, 2006.
- [17] I T Jolliffe and N T Trendafilov and M Uddin, *A Modified Principal Component Technique Based on the LASSO*, Journal of Computational and Graphical Statistics, **12** (3), 531-547, 2003.
- [18] J Schmidt-Hieber, *Nonparametric regression using deep neural networks with ReLU activation function*, The Annals of Statistics, **48**(4), 1875-1897, 2020.
- [19] E Plaut, *From Principal Subspaces to Principal Components with Linear Autoencoders*, arXiv:1804.10253v3 [stat.ML], 2018.
- [20] H Bourland and Y Kamp, *Auto-Association by Multilayer Perceptrons and Singular Value Decomposition*, Biological Cybernetics, **59**, 291-294, 1988.
- [21] I Goodfellow and Y Bengio and A Courville, *Deep Learning* The MIT Press, 18 November 2016.
- [22] DP Kingma and J Ba, *Adam: A Method for Stochastic Optimization*, arXiv:1412.6980 [cs.LG], 2017.
- [23] C Fyfe, *Hebbian learning and negative feedback networks*, Springer International Publishing, 2005.
- [24] C Fyfe, *A Neural Network for PCA and Beyond*, Neural Processing Letters, **6**, 33-41, 1997.
- [25] P Vincent and H Larochelle and I Lajoie and Y Bengio and P Manzagol, *Stacked Denoising Autoencoders: Learning Useful Representations in a Deep Network with a Local Denoising Criterion*, The Journal of Machine Learning Research, **11**, 3378-3408, 2010.
- [26] M Lezcano-Casado and D Martinez-Rubio, *Cheap Orthogonal Constraints in Neural Networks: A Simple Parametrization of the Orthogonal and Unitary Group* arXiv:1901.08428v3 [cs.LG], May 2019.
- [27] P Zhou and XT Yuan and S Yan and J. Feng, *Faster First-Order Methods for Stochastic Non-Convex Optimization on Riemannian Manifolds*, IEEE Transactions on Pattern Analysis and Machine Intelligence, **43**(2), 459-472, 1 Feb. 2021.
- [28] A Paszke and S Gross and F Massa and A Lerer and J Bradbury and G Chanan and S Chintala, *PyTorch: An Imperative Style, High-Performance Deep Learning Library*, Advances in Neural Information Processing Systems, 32, 8024–8035, 2019
- [29] R A Fisher, *The use of multiple measurements in taxonomic problems*, Annals of Eugenics, **7** (2), 179-188, 1936

- [30] "UCI Machine Learning Repository: Iris Data Set", archive.ics.uci.edu, Retrieved 2 June 2025.
- [31] N Elhage and T Hume and C Olsson and N Schiefer and T Henighan and S Kravec and Z Hatfield-Dodds and R Lasenby and D Drain and C Chen and R Grosse and S McCandlish and J Kaplan and D Amodei and M Wattenberg and C Olah, *Toy Models of Superposition*, arXiv:2209.10652 [**cs.LG**], 2022.
- [32] B Ghojogh and A Ghodsi and F Karray and M Crowley, *Factor Analysis, Probabilistic Principal Component Analysis, Variational Inference, and Variational Autoencoder: Tutorial and Survey*, arXiv:2101.00734 [**stat.ML**], May 2022.
- [33] M Moor and M Horn and B Rieck and Borgwardt, *Topological Autoencoders*, arXiv:1906.00722v5 [**cs.LG**], May 2021.
- [34] G Yu and G Sapiro, *Statistical Compressed Sensing of Gaussian Mixture Models*, arXiv:1101.5785 [**cs.CV**], Jan 2011.
- [35] A Grover and S Ermon, *Uncertainty Autoencoders: Learning Compressed Representations via Variational Information Maximization*, arXiv:1812.10539 [**stat.ML**], Apr 2019.
- [36] D C Lay, *Linear Algebra and Its Applications* , Pearson Education, 2012.